

**AI-BASED AUTOMATIC MODULATION
CLASSIFICATION
AND SIGNAL PARAMETER ESTIMATION FOR
WIRELESS COMMUNICATION SYSTEMS**

A MAJOR PROJECT REPORT

Submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

**INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
SENAPATI, MANIPUR**

Submitted by

RAHUL MEENA

Roll No.: 230102069

Academic Session: 2026–27

CERTIFICATE

This is to certify that the major project entitled “AI-Based Automatic Modulation Classification and Signal Parameter Estimation for Wireless Communication Systems” has been carried out by Rahul Meena (Roll No. 230102069), a student of Bachelor of Technology in Electronics and Communication Engineering at the Indian Institute of Information Technology Senapati, Manipur, during the academic session 2026–27.

The work presented in this report has been carried out under the guidance and supervision of the undersigned and is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering.

Project	Supervisor
Name:	_____
Designation:	_____
Signature: _____	

Head	of	Department
Name:		_____
Designation:		_____
Signature: _____		

Date: _____ Place: Senapati, Manipur

DECLARATION

I, Rahul Meena, Roll No. 230102069, student of Bachelor of Technology in Electronics and Communication Engineering at the Indian Institute of Information Technology Senapati, Manipur, hereby declare that the project work entitled “AI-Based Automatic Modulation Classification and Signal Parameter Estimation for Wireless Communication Systems” is my original work carried out during the academic session 2026–27.

The work presented in this report has been developed and documented for the fulfillment of the requirements of the Bachelor of Technology degree. Wherever the work of other researchers, authors, or sources has been used, appropriate references have been provided.

I further declare that this project report has not been submitted, in whole or in part, for the award of any other degree or qualification at any other institution.

Rahul						Meena
Roll			No.:			230102069
B.Tech,	Electronics		and	Communication		Engineering
Indian	Institute	of	Information	Technology	Senapati,	Manipur

Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor for providing valuable guidance, suggestions, and encouragement throughout the development of this major project.

I am grateful to the faculty members of the Department of Electronics and Communication Engineering, Indian Institute of Information Technology Senapati, Manipur, for providing the academic environment and technical knowledge required to undertake this work.

I would also like to thank my institute for providing the facilities and resources necessary for implementing and testing the proposed system.

Finally, I would like to thank my family, friends, and everyone who supported and encouraged me during the development of this project.

Rahul				Meena
B.Tech,	Electronics	and	Communication	Engineering
IIIT Senapati, Manipur				

TABLE OF CONTENTS

Abstract

Chapter 1 — Introduction

Chapter 2 — Literature Review

Chapter 3 — Problem Statement and Objectives

Chapter 4 — Proposed System and System Architecture

Chapter 5 — Methodology

Chapter 6 — Dataset and Experimental Setup

Chapter 7 — Implementation

Chapter 8 — Results and Discussion

Chapter 9 — Streamlit Dashboard

Chapter 10 — Limitations and Future Scope

Chapter 11 — Conclusion

References

Appendix

Note: In Microsoft Word, the table of contents can be updated after opening the document using References → Table of Contents → Update Table.

LIST OF FIGURES

- Figure 1. Overall System Architecture
- Figure 2. Signal Processing Pipeline
- Figure 3. CNN Architecture
- Figure 4. I/Q Signal Representation
- Figure 5. Example Constellation
- Figure 6. Example Frequency Spectrum
- Figure 7. CNN Accuracy vs SNR
- Figure 8. CNN Confusion Matrix
- Figure 9. Modulation Accuracy by Class
- Figure 10. SNR MAE vs SNR
- Figure 11. CFO MAE by CFO Range
- Figure 12. Symbol-Rate Confusion Matrix
- Figure 13. End-to-End Modulation Confusion Matrix
- Figure 14. Final System Performance
- Figure 15. Streamlit Dashboard — Input
- Figure 16. Streamlit Dashboard — AI Analysis
- Figure 17. Streamlit Dashboard — Signal Visualization
- Figure 18. Streamlit Dashboard — Model Performance

LIST OF TABLES

Table 2.1. Comparison of AMC Approaches

Table 6.1. Supported Modulation Classes

Table 6.2. Dataset Parameters

Table 6.3. Software Environment

Table 6.4. Experimental Datasets

Table 7.1. Project Directory Structure

Table 7.2. CNN Architecture

Table 8.1. Overall System Performance

Table 8.2. Modulation Accuracy by Class

Table 8.3. SNR Estimation Performance

Table 8.4. CFO Estimation Performance

Table 8.5. Symbol-Rate Estimation Performance

Table 8.6. End-to-End Evaluation

Table 10.1. Limitations and Future Improvements

AI-Based Automatic Modulation Classification and Signal Parameter Estimation for Wireless Communication Systems

Abstract

Wireless communication systems use different modulation techniques to transmit information efficiently over communication channels. Identifying the modulation scheme and estimating important signal parameters are fundamental tasks in applications such as spectrum monitoring, cognitive radio, wireless security, signal intelligence, and communication-system analysis. Traditional signal analysis methods often depend on manually designed signal-processing algorithms and may become less effective when signals are affected by noise, carrier-frequency offset, and other channel impairments.

This project presents an AI-based system for **Automatic Modulation Classification (AMC)** and **signal parameter estimation** using complex baseband In-phase (I) and Quadrature (Q) samples. The developed system supports eight modulation classes: **BPSK, QPSK, 8PSK, 16QAM, 64QAM, GFSK, AM, and FM**. A one-dimensional Convolutional Neural Network (CNN) is used for modulation classification directly from normalized I/Q signal samples. Separate machine-learning models are used for estimating important signal parameters including **Signal-to-Noise Ratio (SNR), Carrier Frequency Offset (CFO), and symbol rate**.

The system was evaluated using controlled synthetic signals covering multiple modulation schemes, SNR levels, carrier-frequency offsets, and symbol rates. In the final end-to-end evaluation, 1,680 test signals were analyzed. The modulation classifier achieved an overall accuracy of **75.24%**. The SNR estimator achieved a Mean Absolute Error (MAE) of **0.467 dB**, while the CFO estimator achieved an MAE of **688.56 Hz**. The symbol-rate classifier achieved an accuracy of **70.95%** over the supported symbol-rate classes.

A Streamlit-based interactive dashboard was also developed to provide signal generation, I/Q signal upload, AI-based analysis, parameter estimation, waveform visualization, constellation visualization, frequency-spectrum analysis, and performance reporting. The resulting system demonstrates how deep learning and machine learning can be combined with digital signal processing to create an integrated wireless-signal analysis platform.

Keywords: Automatic Modulation Classification, Artificial Intelligence, Machine Learning, Convolutional Neural Network, I/Q Signal, SNR Estimation, Carrier Frequency Offset, Symbol Rate, Wireless Communication, Signal Processing.

Chapter 1 — Introduction

1.1 Background

Wireless communication has become an essential part of modern communication infrastructure. Cellular networks, wireless local-area networks, satellite communication, Internet of Things devices, and radio-based systems use a variety of modulation techniques to transmit information through wireless channels.

Modulation is the process of changing a carrier signal according to the information being transmitted. Different modulation schemes provide different characteristics in terms of bandwidth utilization, spectral efficiency, robustness, and implementation complexity. Examples include Phase Shift Keying (PSK), Quadrature Amplitude Modulation (QAM), Frequency Shift Keying (FSK), Amplitude Modulation (AM), and Frequency Modulation (FM).

In many practical situations, the receiver or monitoring system may not know which modulation scheme is being used. Determining the modulation type from an observed signal is known as **Automatic Modulation Classification (AMC)**.

AMC is useful in several applications, including cognitive radio, spectrum monitoring, wireless communication analysis, signal intelligence, and network monitoring. An AMC system receives signal samples and determines the most probable modulation format without requiring prior manual identification.

In addition to modulation classification, knowledge of signal parameters is important for understanding and analyzing a received wireless signal. Parameters such as Signal-to-Noise Ratio (SNR), Carrier Frequency Offset (CFO), and symbol rate provide information about the quality, frequency alignment, and timing characteristics of the signal.

Artificial Intelligence and Machine Learning provide an alternative approach for automatically learning useful patterns from signal data. Instead of relying entirely on manually designed decision rules, machine-learning models can learn relationships between signal characteristics and their corresponding classes or parameters.

This project therefore combines digital signal processing with machine learning and deep learning to develop an integrated system capable of both **modulation classification and signal parameter estimation**.

1.2 Automatic Modulation Classification

Automatic Modulation Classification is the process of determining the modulation format of a received signal automatically.

The general AMC process can be represented as:

Received I/Q Signal → Preprocessing → Feature Learning → Classification → Modulation Type

In this project, the input is represented as complex baseband I/Q samples:

$$x[n] = I[n] + jQ[n]$$

where:

- $I[n]$ represents the In-phase component.
- $Q[n]$ represents the Quadrature component.
- $j = \sqrt{-1}$.

The system uses both components as input to the CNN. This allows the model to learn temporal characteristics and patterns present in the complex signal.

The developed classifier supports the following eight modulation classes:

Category	Modulation
PSK	BPSK
PSK	QPSK
PSK	8PSK
QAM	16QAM
QAM	64QAM
FSK	GFSK
Analog	AM
Analog	FM

1.3 Signal Parameter Estimation

Identifying the modulation scheme alone does not provide complete information about an observed wireless signal. Important physical and communication parameters must also be estimated.

The developed system estimates three parameters:

Signal-to-Noise Ratio

SNR represents the relationship between the signal power and noise power. It is commonly expressed in decibels:

$$SNR_{dB} = 10 \log_{10} \left(\frac{P_{signal}}{P_{noise}} \right)$$

\]

The project estimates SNR from the received I/Q samples using a trained machine-learning model.

Carrier Frequency Offset

Carrier Frequency Offset occurs when there is a difference between the transmitter and receiver carrier frequencies.

The project estimates CFO in Hertz from the received signal.

The supported evaluation range is approximately:

\[

$$-3000 \leq \text{CFO} \leq 3000 \text{ Hz}$$

\]

Symbol Rate

Symbol rate represents the number of transmitted symbols per second and is measured in baud.

The current system classifies the signal into one of six supported symbol-rate classes:

- 50 kHz
- 76.923 kHz
- 100 kHz
- 125 kHz
- 142.857 kHz
- 200 kHz

1.4 Motivation

Traditional signal-analysis systems may require separate algorithms for different modulation types and signal conditions. Their performance can also depend strongly on manually selected features and assumptions about the received signal.

The motivation of this project is to develop a unified AI-assisted system that can:

1. Automatically classify multiple modulation schemes.

2. Estimate important signal parameters.
3. Operate directly on complex I/Q samples.
4. Handle signals across multiple SNR conditions.
5. Provide an interactive interface for signal analysis.
6. Combine machine learning with conventional signal-processing techniques.
7. Provide measurable performance through controlled experiments.

1.5 Scope of the Project

The scope of the developed system includes:

- Generation of synthetic wireless I/Q signals.
- Support for eight modulation classes.
- Signal normalization and preprocessing.
- CNN-based modulation classification.
- Machine-learning-based SNR estimation.
- Machine-learning-based CFO estimation.
- Symbol-rate classification.
- Evaluation under multiple signal conditions.
- Visualization of signal characteristics.
- Interactive Streamlit dashboard.
- End-to-end performance evaluation.

The current implementation focuses on controlled synthetic signals rather than unrestricted real-world over-the-air recordings.

1.6 Applications

The developed system can serve as a prototype for applications such as:

- Wireless spectrum monitoring
- Cognitive radio

- Software-defined radio analysis
- Communication-system testing
- Signal intelligence research
- Wireless security research
- Educational signal-processing laboratories
- AI-assisted RF signal analysis

1.7 Organization of the Report

The remainder of this report is organized as follows:

Chapter 2 discusses the background and existing approaches related to automatic modulation classification and signal parameter estimation.

Chapter 3 defines the problem statement and project objectives.

Chapter 4 presents the proposed system architecture.

Chapter 5 explains the methodology, including signal generation, preprocessing, CNN-based classification, and parameter estimation.

Chapter 6 describes the datasets and experimental setup.

Chapter 7 presents the implementation of the proposed system and dashboard.

Chapter 8 presents the experimental results and their analysis.

Chapter 9 describes the Streamlit-based user interface.

Chapter 10 discusses the limitations and future scope.

Chapter 11 provides the conclusion of the project.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Automatic Modulation Classification (AMC), also referred to as Automatic Modulation Recognition (AMR), is an important signal-processing task in wireless communication systems. The objective of AMC is to identify the modulation scheme of a received signal without requiring prior knowledge of the transmitter's modulation format. AMC can be useful in applications such as spectrum monitoring, signal identification, cognitive radio, interference monitoring, and wireless communication analysis.

Traditional AMC techniques generally depend on manually designed signal features and statistical or signal-processing methods. With the development of machine learning and deep learning, researchers have increasingly investigated approaches that can learn useful representations directly from received I/Q signal samples.

This chapter reviews major developments relevant to the proposed project, including traditional feature-based approaches, machine-learning-based classification, deep-learning-based AMC, radio signal datasets, and the influence of practical signal impairments.

2.2 Traditional Automatic Modulation Classification

Early AMC systems commonly relied on expert-designed features extracted from received signals. These features can include statistical moments, higher-order moments, spectral characteristics, instantaneous amplitude, instantaneous phase, frequency-related characteristics, and cyclostationary features.

The general processing chain of a traditional AMC system can be represented as:

Received Signal → Preprocessing → Feature Extraction → Classifier → Modulation Decision

The advantage of this approach is that the extracted features can have a direct physical interpretation. However, designing features that remain reliable under different signal-to-noise ratios (SNRs), carrier frequency offsets, symbol rates, fading conditions, and other channel impairments can be difficult.

Cyclostationary and statistical signal-processing techniques have therefore been extensively investigated for signal detection and classification. Such techniques exploit properties of communication signals that distinguish them from noise and from other signal types.

Although feature-engineered methods remain useful, they require substantial domain knowledge and their performance can depend strongly on the selected features and operating conditions.

2.3 Machine Learning Based Modulation Classification

Machine learning introduced the possibility of using automatically learned decision boundaries instead of relying entirely on manually designed classification rules.

A machine-learning AMC system typically extracts a set of signal features and provides them to a classifier such as a decision tree, support vector machine, random forest, or boosting model.

O'Shea and West emphasized the importance of suitable datasets for machine-learning-based radio signal processing. Their work also introduced public synthetic datasets intended to provide common baselines for radio machine-learning experiments and evaluation.

Machine-learning classifiers can provide effective performance when the selected features capture characteristics that distinguish the modulation classes. However, the quality of the feature representation remains an important limitation.

For the proposed project, machine learning is also used for **signal parameter estimation**. Random Forest models are employed for estimating SNR and carrier frequency offset, while a classification model is used for estimating the supported symbol-rate categories.

2.4 Deep Learning for Automatic Modulation Classification

Deep learning changed the AMC research direction by allowing models to learn representations directly from signal samples.

One important early work is “**Convolutional Radio Modulation Recognition Networks**” by O'Shea, Corgan, and Clancy. The authors investigated convolutional neural networks for the complex temporal radio-signal domain and compared learned representations with expert-designed features. Their results demonstrated that convolutional neural networks could learn useful representations for radio modulation recognition directly from temporal signal data.

This approach is particularly relevant to the proposed system because the project uses raw complex I/Q signal samples as the primary input to a CNN.

Instead of manually converting the signal into a fixed set of handcrafted classification features, the proposed CNN receives a two-channel representation:

- Channel 1: In-phase (I) component
- Channel 2: Quadrature (Q) component

The CNN can consequently learn temporal patterns and signal characteristics associated with different modulation schemes.

2.5 Convolutional Neural Networks for Radio Signal Classification

CNNs are particularly suitable for signal classification because convolutional filters can identify local patterns within sequential input data.

For an I/Q signal represented as

$$x[n] = I[n] + jQ[n]$$

the proposed system separates the complex signal into two real-valued sequences:

$$X = \begin{bmatrix} I[0] & I[1] & \cdots & I[N-1] \\ Q[0] & Q[1] & \cdots & Q[N-1] \end{bmatrix}$$

The CNN processes this two-channel representation through multiple convolutional layers.

The architecture implemented in this project contains convolutional blocks followed by batch normalization, ReLU activation, max pooling, adaptive average pooling, and fully connected layers. The final layer produces probabilities for eight modulation classes.

The eight supported classes are:

1. BPSK
2. QPSK
3. 8PSK
4. 16QAM

5. 64QAM

6. GFSK

7. AM

8. FM

A survey of deep-learning-based radio modulation recognition research by Wang et al. reviewed CNN-, RNN-, DBN-, and hybrid-network-based approaches and discussed commonly used datasets and evaluation methods. The survey also identified continuing challenges in robust radio modulation recognition.

2.6 Deep Learning Under Realistic Signal Conditions

An important issue in AMC is that signals encountered in practical wireless environments are affected by various impairments.

O'Shea, Roy, and Clancy investigated deep-learning-based radio signal classification under different configurations and channel impairments. Their study specifically considered effects including **carrier frequency offset, symbol rate, and multipath fading**, and also included over-the-air measurements using software-defined radios.

This work is directly relevant to the present project because the proposed system does not treat modulation classification as an isolated task. The generated signals include variations in:

- SNR
- Carrier frequency offset
- Symbol rate
- Modulation type

The project therefore attempts to evaluate the complete inference pipeline under controlled parameter variations rather than evaluating the classifier only on a single fixed signal configuration.

2.7 Radio Signal Dataset Generation

The availability of suitable datasets is a major requirement for training and evaluating machine-learning models.

O'Shea and West discussed the importance of datasets for radio machine learning and presented synthetic datasets designed to provide common experimental baselines. Their work highlighted

that dataset generation, training, testing, and evaluation are important parts of radio machine-learning research.

The proposed project follows a similar experimental principle by generating controlled synthetic complex I/Q signals.

The project dataset contains eight modulation classes and systematically varies signal parameters. The main dataset configurations include:

- Multiple modulation classes
- Multiple SNR levels
- Multiple symbol rates
- Variable carrier frequency offset
- Fixed sample length of 1024 samples
- Complex I/Q representation

This controlled generation makes it possible to evaluate individual parameters independently as well as evaluate the complete system.

2.8 Signal Parameter Estimation

Most AMC research primarily focuses on determining the modulation type. However, a practical signal-analysis system may also need to estimate other signal parameters.

Important parameters include:

2.8.1 Signal-to-Noise Ratio

SNR represents the relative strength of the desired signal compared with noise.

[

$$\text{SNR} = \frac{P_s}{P_n}$$

]

In decibel form:

[

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_s}{P_n} \right)$$

]

Accurate SNR estimation can assist in determining the reliability of subsequent signal-processing and classification operations.

2.8.2 Carrier Frequency Offset

Carrier frequency offset (CFO) occurs when there is a frequency mismatch between the transmitter and receiver oscillators or when other frequency offsets are present.

A simplified received signal can be represented as:

$$r[n] = s[n]e^{j2\pi f_{\text{CFO}}n/f_s} + w[n]$$

where:

- $s[n]$ is the transmitted signal,
- f_{CFO} is the carrier frequency offset,
- f_s is the sampling frequency,
- $w[n]$ is additive noise.

CFO can affect the phase evolution of a received signal and can consequently influence modulation classification and parameter estimation.

The importance of CFO in radio-signal classification has also been investigated in previous deep-learning-based radio classification research.

2.8.3 Symbol Rate

Symbol rate represents the number of transmitted symbols per second.

It is related to the sampling rate through the samples-per-symbol relationship:

$$\text{SPS} = \frac{f_s}{R_s}$$

where:

- SPS = samples per symbol,
- f_s = sampling frequency,

- $(R_s) = \text{symbol rate}$.

Estimating symbol rate provides additional information about the structure of a received communication signal.

In the proposed system, symbol-rate estimation is implemented as a classification problem over six supported symbol-rate categories.

2.9 Comparison of Existing Approaches

The reviewed approaches can be broadly categorized as follows.

Approach	Main Input	Feature Design	Classifier	Main Characteristic
Traditional statistical AMC	Received signal	Handcrafted statistical features	Statistical/ML classifier	Strong signal-processing dependence
Higher-order feature methods	I/Q signal	Higher-order moments	ML classifier	Compact feature representation
Machine-learning AMC	Extracted signal features	Manually selected	RF/SVM/boosting etc.	Depends on feature quality
CNN-based AMC	Raw/processed I/Q	Learned automatically	CNN	Learns temporal signal representations
Hybrid/advanced DL	I/Q or transformed signal	Learned + engineered	CNN/RNN/hybrid	More complex representation learning
Proposed system	Complex I/Q	CNN + engineered parameter features	CNN + Random Forest/classifier	AMC combined with parameter estimation

The literature shows a general transition from manually engineered signal features toward learned representations using deep neural networks.

2.10 Research Gap

The reviewed literature demonstrates substantial progress in automatic modulation recognition, particularly through machine learning and deep learning.

However, several practical issues remain relevant:

1. **Modulation classification is often treated as the primary task**, while other signal parameters may be handled separately.
2. **Realistic signal impairments can affect classification performance.** Carrier frequency offset, symbol rate variation, multipath, and noise can change the characteristics observed by a classifier.

3. ****Dataset design strongly influences experimental results.**** Controlled and reproducible datasets are therefore important for evaluating models.
4. ****A classification result alone does not provide a complete description of an unknown signal.**** Additional estimates such as SNR, CFO, and symbol rate can provide useful signal-level information.

Based on these observations, the present project combines several related tasks into a single prototype system:

I/Q Signal → Modulation Classification + SNR Estimation + CFO Estimation + Symbol-Rate Estimation → Interactive Analysis Dashboard

The project therefore focuses not only on identifying the modulation class but also on estimating selected signal parameters from the same received signal representation.

The project should be considered an integrated prototype rather than a claim of being the first system to perform these tasks. Its main contribution is the implementation and experimental integration of these components within one reproducible system.

2.11 Summary

This chapter reviewed the development of automatic modulation classification from traditional feature-based methods to machine-learning and deep-learning approaches.

Previous research has demonstrated that convolutional neural networks can learn useful representations directly from radio signal samples, while subsequent work has investigated performance under practical impairments such as carrier frequency offset and varying symbol rates. Dataset generation has also been identified as an important component of reproducible radio machine-learning research.

Based on these developments, the proposed project uses a CNN for modulation classification and machine-learning models for signal parameter estimation. The resulting system integrates modulation recognition, SNR estimation, CFO estimation, symbol-rate estimation, visualization, and interactive analysis within a single software framework.

References Used in This Chapter

- [1] T. J. O'Shea, J. Corgan, and T. C. Clancy, "Convolutional Radio Modulation Recognition Networks," *Engineering Applications of Neural Networks (EANN)*, 2016, pp. 213–226, doi: 10.1007/978-3-319-44188-7_16.
- [2] T. J. O'Shea and N. West, "Radio Machine Learning Dataset Generation with GNU Radio," *Proceedings of the GNU Radio Conference*, vol. 1, no. 1, 2016.

- [3] T. J. O'Shea, T. Roy, and T. C. Clancy, "Over-the-Air Deep Learning Based Radio Signal Classification," *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 168–179, 2018, doi: 10.1109/JSTSP.2018.2797022.
- [4] T. Wang, G. Yang, P. Chen, Z. Xu, and M. Jiang, "A Survey of Applications of Deep Learning in Radio Signal Modulation Recognition," *Applied Sciences*, vol. 12, no. 23, 12052, 2022.

Chapter 3 — Problem Statement and Objectives

3.1 Problem Statement

Wireless communication systems operate using different modulation techniques and may experience various impairments such as noise and carrier-frequency mismatch. For effective signal monitoring and analysis, it is important to identify the modulation scheme and estimate relevant signal parameters from received signal samples.

Conventional signal-analysis approaches often use individually designed signal-processing algorithms and manually selected features for different tasks. Such approaches can become difficult to maintain when multiple modulation schemes and varying signal conditions need to be handled within a single system.

The problem addressed in this project is therefore the development of an **AI-based integrated wireless signal analysis system** capable of automatically classifying multiple modulation schemes and estimating important signal parameters from complex baseband I/Q samples.

The developed system addresses four primary analysis tasks:

1. Automatic classification of the modulation type.
2. Estimation of Signal-to-Noise Ratio.
3. Estimation of Carrier Frequency Offset.
4. Classification of the symbol rate among predefined supported rates.

The system should also provide an accessible interface through which users can generate or upload I/Q signals and inspect the resulting AI predictions and signal characteristics.

3.2 Existing System

Traditional Automatic Modulation Classification systems generally rely on signal-processing techniques such as:

- Statistical feature extraction
- Higher-order statistics
- Spectral analysis
- Cyclostationary analysis
- Likelihood-based methods

- Expert-designed decision rules

These methods can provide useful classification performance under specific assumptions. However, their effectiveness may depend on the selected features, signal conditions, noise level, and prior knowledge of the communication system.

Machine-learning approaches can reduce the dependence on manually designed decision rules by learning relationships between signal characteristics and target classes from training data.

Deep-learning approaches extend this concept by allowing neural networks to learn hierarchical representations directly from signal samples.

3.3 Proposed System

The proposed system combines digital signal processing, machine learning, and deep learning into an integrated wireless signal analysis platform.

The overall processing flow is:

I/Q Signal → Preprocessing → AI Models → Modulation and Parameter Estimation → Visualization

The system accepts complex baseband I/Q samples either from the built-in signal generator or from an uploaded signal file.

For modulation classification, the I/Q samples are normalized and supplied to a one-dimensional Convolutional Neural Network. The CNN learns temporal patterns in the I/Q data and predicts one of the eight supported modulation classes.

For parameter estimation, dedicated machine-learning models process extracted signal characteristics and estimate SNR, CFO, and symbol rate.

The main components of the proposed system are:

1. Signal Generation

Synthetic I/Q signals are generated with configurable:

- Modulation type
- SNR
- Carrier-frequency offset
- Symbol rate
- Number of samples

- Random seed

2. Signal Preprocessing

The generated or uploaded complex signal is converted into its I/Q representation and normalized before being supplied to the appropriate AI model.

3. Modulation Classification

A one-dimensional CNN is used to classify:

- BPSK
- QPSK
- 8PSK
- 16QAM
- 64QAM
- GFSK
- AM
- FM

4. SNR Estimation

A trained Random Forest regression model estimates the SNR of the input signal.

5. CFO Estimation

A trained Random Forest regression model estimates the carrier-frequency offset in Hertz.

6. Symbol-Rate Classification

A trained Random Forest classifier determines the most probable symbol-rate class from the supported set of six rates.

7. Visualization and Dashboard

A Streamlit dashboard displays:

- Predicted modulation
- Modulation confidence
- Estimated SNR

- Estimated CFO
- Estimated symbol rate
- I/Q waveform
- Constellation
- Frequency spectrum
- Amplitude and phase
- Model performance
- Analysis report

3.4 Project Objectives

The primary objective of this project is to develop an AI-based system for automatic modulation classification and signal parameter estimation from complex I/Q samples.

The specific objectives are:

Objective 1 — Automatic Modulation Classification

Develop a CNN-based classifier capable of identifying eight different modulation schemes from complex I/Q samples.

Objective 2 — SNR Estimation

Develop a machine-learning-based model capable of estimating the Signal-to-Noise Ratio of the received signal.

Objective 3 — CFO Estimation

Develop a machine-learning-based model for estimating Carrier Frequency Offset over the defined operating range.

Objective 4 — Symbol-Rate Classification

Develop a machine-learning model capable of identifying the symbol-rate class from a predefined set of supported rates.

Objective 5 — Synthetic Signal Dataset

Generate a controlled dataset containing signals with different modulation types, SNR conditions, carrier-frequency offsets, and symbol rates.

Objective 6 — End-to-End Evaluation

Evaluate the complete system using a controlled test set and calculate appropriate performance metrics for each analysis task.

Objective 7 — Interactive Analysis Platform

Develop a Streamlit dashboard that allows users to generate or upload I/Q signals and view AI-based signal analysis results.

Objective 8 — Signal Visualization

Provide visualization tools for examining important signal characteristics, including time-domain I/Q waveforms, constellation diagrams, frequency spectra, amplitude, and phase.

3.5 Functional Requirements

The system shall provide the following functionality:

Requirement	Description
Signal Generation	Generate synthetic I/Q signals using configurable parameters.
Signal Upload	Accept supported I/Q signal file formats including `.npy`, `.npz`, and `.csv`.
Preprocessing	Normalize and prepare I/Q samples for model inference.
Modulation Classification	Predict one of eight supported modulation classes.
SNR Estimation	Estimate SNR in decibels.
CFO Estimation	Estimate carrier-frequency offset in Hertz.
Symbol-Rate Classification	Predict one of six supported symbol-rate classes.
Visualization	Display waveform, constellation, spectrum, amplitude, and phase.
Performance Analysis	Display evaluated model performance.
Report Export	Provide an analysis-report export function.

3.6 Non-Functional Requirements

Accuracy

The system should provide measurable and reproducible performance for the supported signal conditions.

Reliability

The system should validate input signals and reject invalid or non-finite data.

Usability

The dashboard should provide a simple interface for generating, uploading, analyzing, and visualizing signals.

Reproducibility

Synthetic signal generation and model evaluation should support controlled random seeds and documented datasets.

Maintainability

The system should separate signal generation, preprocessing, feature extraction, model training, inference, and visualization into modular components.

Extensibility

The architecture should allow additional modulation schemes, signal conditions, datasets, and estimation models to be incorporated in future development.

3.7 Expected Outcomes

The expected outcome of the project is an integrated prototype capable of analyzing complex wireless I/Q signals using AI-based techniques.

The final implemented system provides:

- Classification of eight modulation types.
- SNR estimation.
- CFO estimation.
- Symbol-rate classification for six supported rates.
- End-to-end evaluation across multiple signal conditions.
- Interactive signal visualization.
- An AI-based Streamlit analysis dashboard.

The final system achieved the following measured end-to-end performance:

Parameter	Performance
Modulation Classification Accuracy	75.24%
SNR Mean Absolute Error	0.467 dB
CFO Mean Absolute Error	688.56 Hz
Symbol-Rate Classification Accuracy	70.95%
End-to-End Test Signals	1,680

These results provide a quantitative basis for evaluating the developed system and identifying areas for future improvement.

Chapter 4 — Proposed System and Architecture

4.1 Introduction

The proposed system is an AI-based wireless signal analysis platform designed to perform Automatic Modulation Classification and estimation of important signal parameters from complex baseband I/Q samples.

The system combines signal generation, signal preprocessing, deep-learning-based modulation classification, machine-learning-based parameter estimation, visualization, and an interactive dashboard into a single framework.

The major processing stages are:

Input I/Q Signal → Preprocessing → AI Analysis → Parameter Estimation → Visualization → Results

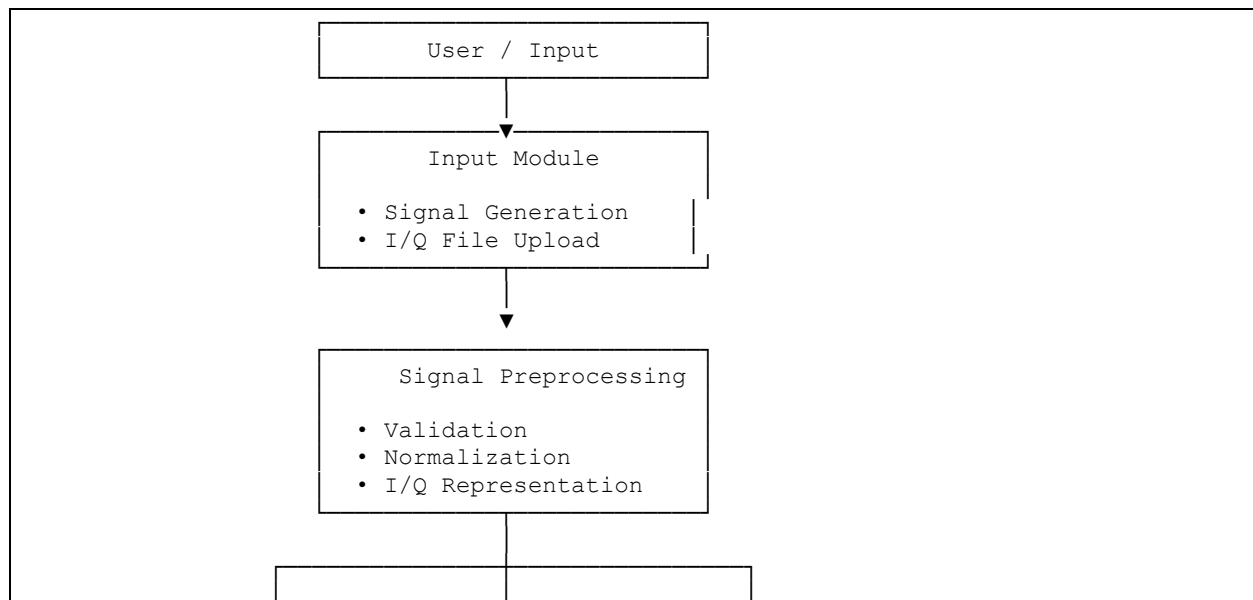
The system can operate in two input modes:

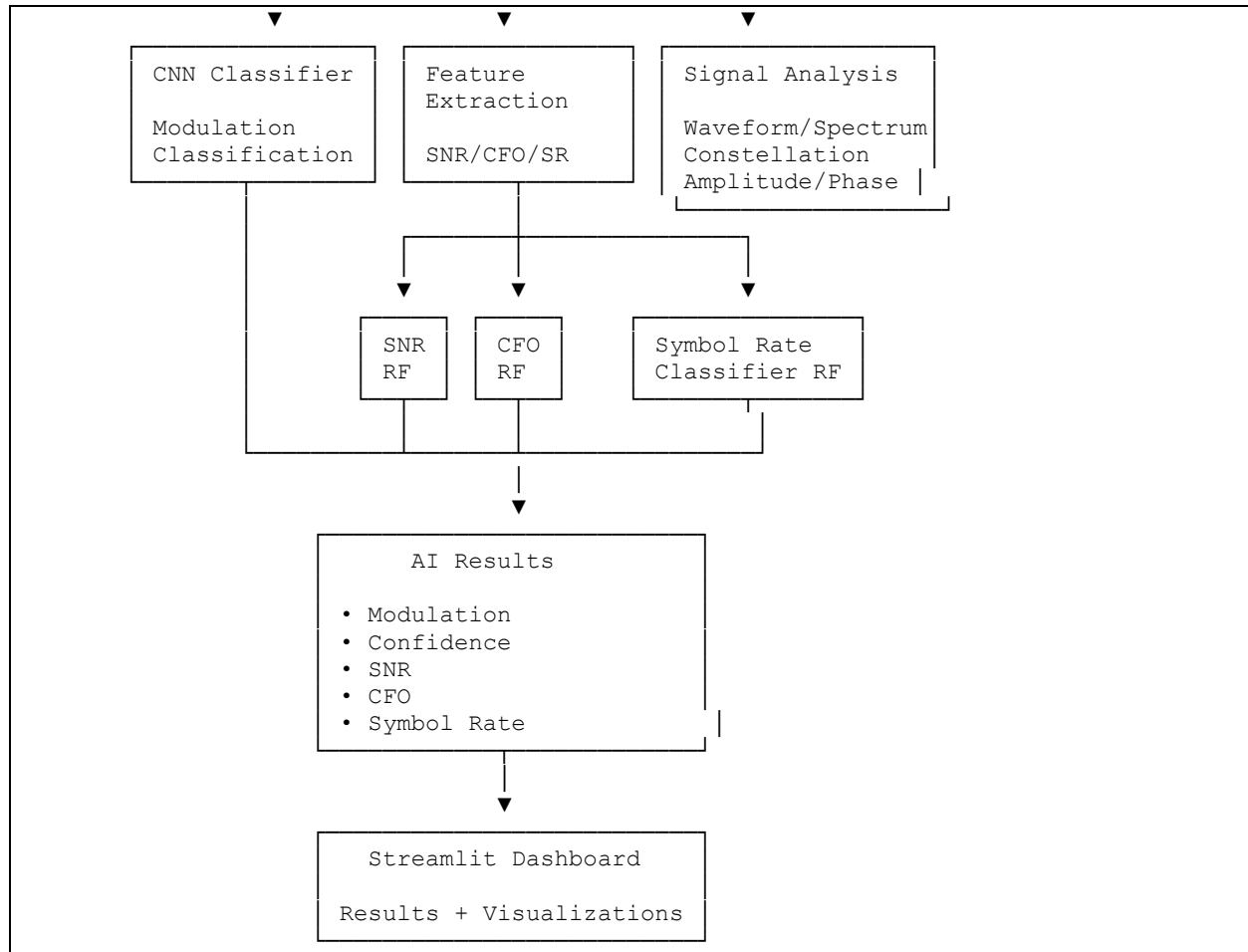
1. ****Synthetic Signal Generation****
2. ****I/Q Signal Upload****

For synthetic signals, the user can specify the signal characteristics before analysis. For uploaded signals, the dashboard validates and processes the supplied I/Q data.

4.2 Overall System Architecture

The overall architecture of the proposed system consists of the following major components:





4.3 Input Module

The input module provides two methods for supplying signals to the system.

4.3.1 Synthetic Signal Generation

The system contains a configurable signal generator for creating controlled complex baseband signals.

The user can select:

- Modulation type
- SNR
- Carrier Frequency Offset
- Symbol rate

- Number of samples
- Random seed

The generated signal is represented as:

$$x[n] = I[n] + jQ[n]$$

where $I[n]$ and $Q[n]$ are the in-phase and quadrature components.

The generator supports the eight modulation classes used by the project:

- BPSK
- QPSK
- 8PSK
- 16QAM
- 64QAM
- GFSK
- AM
- FM

Controlled signal generation allows the system to evaluate model performance under known signal conditions.

4.4 I/Q Signal Upload

The dashboard also supports analysis of externally supplied I/Q signals.

The implemented system accepts:

- `.npy`
- `.npz`
- `.csv`

For CSV files, the system supports complex-valued data or separate I/Q columns where applicable.

Input validation is performed before inference. The system checks for invalid or non-finite signal values and prevents invalid input from being passed to the models.

This provides a flexible interface for testing signals other than those generated internally.

4.5 Signal Preprocessing

Preprocessing prepares the received signal for AI-based analysis.

The main preprocessing operations include:

4.5.1 Conversion to Complex Representation

The signal is represented in complex baseband form:

$$x[n] = I[n] + jQ[n]$$

The real part corresponds to $I[n]$, while the imaginary part corresponds to $Q[n]$.

4.5.2 Mean Removal

The preprocessing stage removes the mean of the complex signal:

$$x_c[n] = x[n] - \mu_x$$

where μ_x is the mean of the input signal.

4.5.3 Normalization

The signal is normalized using its power:

$$x_{\text{norm}}[n] = \frac{x_c[n]}{\sqrt{\mathbb{E}[|x_c[n]|^2] + \epsilon}}$$

where ϵ is a small value used to avoid division by zero.

For CNN input, the real and imaginary components are then arranged as two channels:

$$X = \begin{bmatrix} I[0] & I[1] & \cdots & I[N-1] \\ Q[0] & Q[1] & \cdots & Q[N-1] \end{bmatrix}$$

For the implemented CNN, the input shape is:

$$2 \times 1024$$

4.6 CNN-Based Modulation Classification

The modulation classifier uses a one-dimensional Convolutional Neural Network.

Unlike a conventional machine-learning classifier that depends primarily on manually extracted features, the CNN receives the normalized I/Q sequence and learns useful signal representations through convolutional layers.

The CNN architecture consists of:

Layer	Configuration
Input	2×1024 I/Q samples
Convolution 1	$2 \rightarrow 32$ channels
Batch Normalization	Applied
Activation	ReLU
Pooling	Max Pooling
Convolution 2	$32 \rightarrow 64$ channels
Batch Normalization	Applied
Activation	ReLU
Pooling	Max Pooling
Convolution 3	$64 \rightarrow 128$ channels
Batch Normalization	Applied
Activation	ReLU
Pooling	Max Pooling

Convolution 4	128 $\rightarrow$ 128 channels
Batch Normalization	Applied
Activation	ReLU
Global Pooling	Adaptive Average Pooling
Fully Connected	128 $\rightarrow$ 64
Dropout	Applied
Output	64 $\rightarrow$ 8

The final layer produces eight class scores corresponding to the eight supported modulation schemes.

A probability distribution is obtained from the output scores, and the class with the highest probability is selected as the predicted modulation.

4.7 SNR Estimation

SNR estimation is performed using a dedicated Random Forest regression model.

Signal characteristics are extracted from the input I/Q samples and supplied to the trained model.

The model produces a continuous SNR estimate in decibels.

The final selected SNR model was trained using signals with varying:

- Modulation types
- SNR values
- Carrier-frequency offsets
- Signal conditions

The final end-to-end evaluation produced an SNR MAE of:

$$\boxed{0.467 \text{ dB}}$$

This indicates that the estimated SNR is generally close to the corresponding controlled test value within the evaluated dataset.

4.8 CFO Estimation

Carrier Frequency Offset estimation is performed using a dedicated Random Forest regression model.

CFO represents the frequency difference between the expected carrier frequency and the observed carrier frequency.

The project evaluates CFO over an approximate range of:

$$[-3000 \text{ Hz} \leq \text{CFO} \leq 3000 \text{ Hz}]$$

The system extracts frequency- and phase-related characteristics from the signal and uses them as inputs to the trained regression model.

The final end-to-end evaluation produced:

$$\boxed{\text{MAE} = 688.56 \text{ Hz}}$$

CFO estimation is more challenging than SNR estimation in the current implementation. The evaluation showed larger errors toward the extremes of the supported CFO range.

4.9 Symbol-Rate Classification

The system estimates symbol rate using a dedicated Random Forest classifier.

The current implementation supports six predefined symbol-rate classes:

Class	Symbol Rate
1	50 kHz
2	76.923 kHz
3	100 kHz
4	125 kHz
5	142.857 kHz
6	200 kHz

Signal features related to periodicity, spectrum, amplitude, phase, and other signal characteristics are extracted before classification.

The model predicts the most probable supported symbol-rate class.

The final end-to-end symbol-rate classification accuracy was:

$$[$$

\boxed{70.95\%}

\]

The system therefore performs classification among the predefined supported rates rather than unrestricted continuous symbol-rate estimation.

4.10 Feature Extraction

Separate feature-extraction pipelines are used for parameter estimation.

The implemented feature extractors generate feature vectors of different sizes:

Task	Feature Vector Size
SNR Estimation	15
CFO Estimation	30
Symbol-Rate Classification	40

The extracted characteristics include statistical, amplitude, phase, spectral, and signal-structure information.

This allows the parameter-estimation models to use compact numerical representations rather than processing the complete I/Q sequence directly.

4.11 Model Inference Pipeline

During inference, all trained models are loaded by the inference module.

The final model set consists of:

```
models/  
├─ cnn_common.pth  
├─ snr_parameter_rf.joblib  
├─ cfo_parameter_rf.joblib  
└─ symbol_rate_classifier.joblib
```

For an input signal, the inference pipeline performs:

1. Input validation.
2. Signal preprocessing.
3. CNN-based modulation classification.
4. Feature extraction.
5. SNR prediction.

6. CFO prediction.
7. Symbol-rate classification.
8. Result aggregation.
9. Visualization and dashboard presentation.

The resulting output contains:

- Predicted modulation
- Modulation confidence
- Modulation probabilities
- Estimated SNR
- Estimated CFO
- Estimated symbol rate
- Sample rate
- Number of analyzed samples

4.12 End-to-End Processing

The complete system operates as follows:

Step 1 — Signal Input

The user generates a signal or uploads an I/Q signal.

Step 2 — Validation

The system verifies that the input contains valid signal samples.

Step 3 — Preprocessing

The signal is normalized and converted into the required representation.

Step 4 — Modulation Classification

The normalized I/Q signal is passed to the CNN.

Step 5 — Parameter Feature Extraction

Signal features are calculated for the parameter-estimation models.

Step 6 — Parameter Estimation

The trained Random Forest models estimate SNR and CFO, while the symbol-rate classifier predicts the supported symbol-rate class.

Step 7 — Result Aggregation

All predictions are combined into a single analysis result.

Step 8 — Visualization

The dashboard displays the signal and analysis results through multiple visualizations.

4.13 Dashboard Architecture

The Streamlit dashboard acts as the user-facing layer of the system.

The dashboard provides:

Input Controls

- Signal-generation parameters
- File upload
- Analysis trigger

AI Analysis

- Modulation prediction
- Confidence
- SNR
- CFO
- Symbol rate

Signal Visualization

- I/Q waveform
- Constellation
- Frequency spectrum
- Amplitude

- Phase

Performance Visualization

- Overall modulation accuracy
- Accuracy by modulation class
- SNR performance
- CFO performance
- Symbol-rate confusion matrix
- Modulation confusion matrix

Report Generation

The dashboard also provides an option to export the signal-analysis results as a report.

4.14 System Architecture Summary

The proposed architecture separates the complete wireless-signal analysis task into specialized components.

The CNN is responsible for learning temporal I/Q patterns for modulation classification, while dedicated feature-based Random Forest models perform signal-parameter estimation.

This modular architecture makes it possible to independently train, evaluate, and improve each component while maintaining a unified inference and visualization interface.

The architecture also provides a foundation for future extensions such as additional modulation classes, real-world RF datasets, improved parameter estimators, and more advanced deep-learning models.

Chapter 5 — Methodology

5.1 Introduction

The methodology of the proposed system consists of signal generation, preprocessing, feature extraction, model training, model evaluation, and real-time inference. The complete methodology combines digital signal processing techniques with machine-learning and deep-learning models.

The developed system uses two complementary approaches:

1. ****Deep learning directly from I/Q samples**** for Automatic Modulation Classification.
2. ****Feature-based machine learning**** for SNR estimation, CFO estimation, and symbol-rate classification.

The complete methodology can be represented as:

Signal Generation/Upload → Preprocessing → Feature Extraction → Model Inference → Parameter Estimation → Result Visualization

5.2 Signal Representation

The wireless signal is represented using complex baseband samples.

A complex sample is expressed as:

$$\begin{aligned} & \backslash[\\ x[n] &= I[n] + jQ[n] \\ & \backslash] \end{aligned}$$

where:

- $I[n]$ is the In-phase component.
- $Q[n]$ is the Quadrature component.
- j is the imaginary unit.
- n represents the sample index.

Using complex I/Q samples allows both amplitude and phase information to be represented.

The system processes signals using a fixed analysis length of **1024 samples** for the primary CNN inference pipeline.

5.3 Synthetic Signal Generation

A controlled synthetic signal generator was developed to create training and evaluation data.

The generator supports the following modulation classes:

- BPSK
- QPSK
- 8PSK
- 16QAM
- 64QAM
- GFSK
- AM
- FM

The signal generator allows the following parameters to be controlled:

- Number of samples
- SNR
- Sample rate
- Symbol rate
- Carrier Frequency Offset
- Random seed

Controlled generation provides known ground-truth parameters, which are required for supervised training and quantitative evaluation.

5.4 Symbol Generation

For digital modulation schemes, information symbols are generated according to the selected modulation format.

5.4.1 BPSK

BPSK uses two phase states. The generated symbols can be represented by:

$\sqrt{E_s}$

$$s_k \in \{-1, +1\}$$

\]

5.4.2 PSK

For M-ary PSK modulation, the symbols are represented as:

\[

$$s_k = e^{j2\pi m/M}$$

\]

where:

- M is the number of phase states.
- $m \in \{0, 1, \dots, M-1\}$.

This representation is used for QPSK and 8PSK.

5.4.3 QAM

For QAM signals, real and imaginary symbol components are selected from discrete amplitude levels.

The resulting complex symbol is:

\[

$$s_k = I_k + jQ_k$$

\]

This approach is used to generate 16QAM and 64QAM signals.

5.5 Samples per Symbol

The relationship between sample rate and symbol rate determines the number of samples representing each symbol.

The samples-per-symbol value is calculated approximately as:

\[

$$SPS = \frac{f_s}{R_s}$$

\]

where:

- f_s is the sample rate.
- R_s is the symbol rate.

For the implemented signal generator, the samples-per-symbol value is converted to an integer for signal construction.

The resulting symbols are repeated according to the selected samples-per-symbol value to construct the discrete-time waveform.

5.6 Carrier Frequency Offset

Carrier Frequency Offset is introduced by applying a frequency-dependent phase rotation.

The signal is multiplied by:

$$e^{j2\pi f_{\text{CFO}} t}$$

where:

- f_{CFO} is the carrier frequency offset.
- t is the sampling-time vector.

Therefore, the signal containing CFO can be represented approximately as:

$$x_{\text{CFO}}[n] = x[n] e^{j2\pi f_{\text{CFO}} t_n}$$

The project evaluates CFO values within an approximate range of:

$$[-3000, 3000] \text{ Hz}$$

5.7 Noise Addition

Additive noise is introduced to simulate communication-channel degradation.

The noise power is controlled using the selected SNR value.

For a target SNR expressed in decibels:

$$P_{\text{noise}} = 10^{\{-\text{SNR}_{\text{dB}}\}/10}$$

For complex Gaussian noise, independent real and imaginary components are generated and scaled according to the required noise power.

The received signal is therefore represented as:

$$x_{\text{received}}[n] = x[n] + w[n]$$

where $w[n]$ represents complex noise.

The use of multiple SNR levels allows the models to be evaluated under different noise conditions.

5.8 Signal Normalization

Before CNN inference, the complex signal is normalized.

First, the mean signal value is removed:

$$x_c[n] = x[n] - \mu_x$$

where:

$$\mu_x = \frac{1}{N} \sum_{n=0}^{N-1} x[n]$$

The signal is then normalized using its root mean-square magnitude:

$$x_{\text{norm}}[n] = \frac{x_c[n]}{\sqrt{\frac{1}{N} \sum_{n=0}^{N-1} |x_c[n]|^2 + \epsilon}}$$

where ϵ is a small constant used to prevent division by zero.

For CNN input, the normalized complex signal is separated into its real and imaginary components.

5.9 I/Q Tensor Formation

The CNN receives the I/Q signal as two input channels.

For a signal containing 1024 samples:

$$X = \begin{bmatrix} I_1 & I_2 & \cdots & I_{1024} \\ Q_1 & Q_2 & \cdots & Q_{1024} \end{bmatrix}$$

Thus, each signal is represented using a tensor of shape:

$$2 \times 1024$$

During batch processing, the tensor has the form:

$$[$$

$(B, 2, 1024)$

]

where B represents the batch size.

5.10 CNN-Based Modulation Classification

A one-dimensional CNN is used to classify the modulation type.

The CNN processes the temporal structure of the I/Q samples through a sequence of convolutional layers.

The implemented architecture contains four convolutional stages.

First Convolutional Block

The first layer transforms the two I/Q channels into 32 learned feature channels.

The block contains:

- 1D convolution
- Batch normalization
- ReLU activation
- Max pooling

Second Convolutional Block

The second convolutional layer increases the representation from 32 to 64 channels.

The block again uses:

- 1D convolution
- Batch normalization
- ReLU
- Max pooling

Third Convolutional Block

The third convolutional layer increases the representation from 64 to 128 channels.

It uses:

- 1D convolution
- Batch normalization
- ReLU
- Max pooling

Fourth Convolutional Block

The fourth convolutional layer maintains 128 feature channels and applies:

- 1D convolution
- Batch normalization
- ReLU

An adaptive average-pooling layer then produces a fixed-size representation.

5.11 Fully Connected Classification Layer

The learned representation from the convolutional layers is passed to fully connected layers.

The classifier contains:

$$\begin{bmatrix} 128 \\ \rightarrow 64 \\ \rightarrow 8 \end{bmatrix}$$

The first fully connected layer reduces the learned representation to 64 units.

Dropout is used to reduce overfitting.

The final layer contains eight output units corresponding to the eight modulation classes.

The predicted modulation is obtained from the class with the highest output probability.

5.12 Modulation Classification Training

The CNN is trained using labeled I/Q signals.

Each training example contains:

- I/Q samples
- Corresponding modulation label

The dataset is divided into training and testing subsets.

The training process uses:

- Adam optimizer
- Learning rate of (10^{-3})
- Batch size of 64
- Cross-entropy classification loss

The CNN is trained for multiple epochs while monitoring model performance.

The final common CNN model is saved as:

```
models/cnn_common.pth
```

5.13 Feature Extraction for Parameter Estimation

Unlike the CNN classifier, the parameter-estimation models use numerical signal features.

Three feature extraction pipelines were developed:

Estimation Task	Number of Features
SNR	15
CFO	30
Symbol Rate	40

The extracted features include characteristics related to:

- I/Q statistics
- Amplitude
- Phase
- Phase differences
- Frequency-domain characteristics
- Spectral power
- Signal distribution
- Periodicity

The resulting feature vectors are provided to dedicated Random Forest models.

5.14 SNR Estimation Methodology

The SNR estimation model is formulated as a regression problem.

The extracted 15-dimensional feature vector is given to a Random Forest regression model.

The model learns a mapping:

$$\begin{aligned} & \backslash[\\ & F_{\{SNR\}}:X_{\{features\}} \rightarrow SNR \\ & \backslash] \end{aligned}$$

where $(X_{\{features\}})$ represents the extracted signal characteristics.

The final model is stored as:

`models/snr_parameter_rf.joblib`

The model is evaluated using:

- Mean Absolute Error
- Root Mean Squared Error
- (R^2) score

5.15 CFO Estimation Methodology

CFO estimation is also formulated as a regression problem.

Thirty signal features are extracted from the input signal.

The Random Forest model learns:

$$\begin{aligned} & \backslash[\\ & F_{\{CFO\}}:X_{\{features\}} \rightarrow CFO \\ & \backslash] \end{aligned}$$

The final model is stored as:

`models/cfo_parameter_rf.joblib`

The model output is expressed in Hertz.

The evaluation uses:

- Mean Absolute Error
- Root Mean Squared Error
- (R^2) score
- Correlation analysis
- Error analysis across different CFO ranges

5.16 Symbol-Rate Classification Methodology

Symbol-rate estimation is implemented as a classification problem rather than unrestricted continuous regression.

The model identifies one of six supported symbol-rate classes:

[
{50k, 76.923k, 100k, 125k, 142.857k, 200k} Hz
]

A 40-dimensional signal feature vector is provided to a Random Forest classifier.

The trained model is stored as:

```
models/symbol_rate_classifier.joblib
```

The model is evaluated using classification accuracy and additional regression-style error metrics obtained by comparing the predicted class rate with the ground-truth rate.

5.17 Dataset Construction

Multiple datasets were generated during development for model development and evaluation.

The final parameter dataset contains:

- 33,600 signals
- 8 modulation classes
- Multiple SNR conditions
- 6 symbol-rate conditions

- Multiple CFO values
- 1024 samples per signal

A common train-test split was created to ensure consistent evaluation.

The final common parameter dataset contains:

Dataset	Samples
Training	23,520
Testing	10,080

The end-to-end evaluation uses a separate controlled evaluation configuration containing:

[

8×7×6×5=1680

]

test signals.

These represent combinations of:

- 8 modulation classes
- 7 SNR levels
- 6 symbol rates
- 5 repeated test signals

5.18 SNR Conditions

The SNR values used for the final parameter datasets include:

[

-10, -5, 0, 5, 10, 15, 20 dB

]

This allows the system to be tested from low-SNR conditions to relatively clean signals.

The modulation-classification model was therefore evaluated across a broad range of signal-quality conditions.

5.19 Model Evaluation Metrics

Different metrics are used according to the type of prediction task.

Classification Accuracy

For classification:

$$\begin{aligned} & \backslash[\\ & \text{Accuracy=} \\ & \backslash\frac{\{N_{\text{correct}}\}}{\{N_{\text{total}}\}} \\ & \backslash] \end{aligned}$$

Accuracy is used for modulation classification and symbol-rate classification.

Mean Absolute Error

For parameter estimation:

$$\begin{aligned} & \backslash[\\ & \text{MAE=} \\ & \backslash\frac{1}{N}\sum_{i=1}^N|y_i-\hat{y}_i| \\ & \backslash] \end{aligned}$$

where:

- y_i is the true value.
- $\hat{y}_i$ is the predicted value.

Root Mean Squared Error

$$\begin{aligned} & \backslash[\\ & \text{RMSE=} \\ & \backslash\sqrt{\frac{1}{N}\sum_{i=1}^N(y_i-\hat{y}_i)^2} \\ & \backslash] \end{aligned}$$

\]

RMSE gives greater weight to larger prediction errors.

Coefficient of Determination

The (R^2) score is used to measure how much of the variation in the target variable is explained by the regression model.

5.20 End-to-End Inference

The final inference process integrates all trained models.

For each input signal:

1. The signal is received.
2. The input is validated.
3. The signal is normalized.
4. I/Q channels are prepared for the CNN.
5. The CNN predicts the modulation.
6. Signal features are extracted.
7. The SNR model predicts SNR.
8. The CFO model predicts carrier-frequency offset.
9. The symbol-rate classifier predicts the supported symbol-rate class.
10. The predictions are combined into a single result.
11. Signal visualizations are generated.
12. Results are displayed in the dashboard.

The final inference output contains:

Modulation
Modulation Confidence
Modulation Probabilities
SNR
CFO
Symbol Rate
Sample Rate
Number of Samples

5.21 Methodology Summary

The developed methodology combines direct deep learning from I/Q samples with feature-based machine learning.

The CNN learns modulation-specific patterns from the temporal I/Q representation, while the Random Forest models use engineered signal features for parameter estimation.

This separation allows each model to focus on a specific prediction task and makes the complete system modular.

The final methodology provides an end-to-end pipeline from wireless I/Q signal acquisition to AI-based classification, parameter estimation, visualization, and reporting.

Chapter 6 — Dataset and Experimental Setup

6.1 Introduction

The performance of an AI-based signal analysis system depends strongly on the quality, diversity, and organization of the training and evaluation data. Since the objective of this project is to classify modulation schemes and estimate signal parameters under controlled conditions, synthetic complex I/Q signals were generated with known ground-truth parameters.

The experimental setup was designed to evaluate the system across multiple modulation types, SNR levels, carrier-frequency offsets, and symbol rates.

6.2 Modulation Classes

The dataset contains eight modulation classes:

No.	Modulation	Type
1	BPSK	Digital PSK
2	QPSK	Digital PSK
3	8PSK	Digital PSK
4	16QAM	Digital QAM
5	64QAM	Digital QAM
6	GFSK	Digital FSK
7	AM	Analog
8	FM	Analog

These classes provide a mixture of phase-, amplitude-, and frequency-based signal characteristics.

6.3 Signal Length

The primary models operate on signals containing:

$\backslash[$

$N=1024$

$\backslash]$

complex I/Q samples.

Each sample contains two components:

$\backslash[$

$x[n]=I[n]+jQ[n]$

\]

Therefore, the CNN receives each signal as a two-channel sequence with dimensions:

\[

2×1024

\]

This representation preserves both components of the complex baseband signal.

6.4 Sampling Configuration

The primary signal generation configuration uses a sample rate of:

\[

$f_s = 1 \text{ MHz}$

\]

The symbol rate is varied during dataset generation to produce different samples-per-symbol conditions.

The supported symbol rates are:

No.	Symbol Rate
1	50,000 Hz
2	76,923.08 Hz
3	100,000 Hz
4	125,000 Hz
5	142,857.14 Hz
6	200,000 Hz

These values correspond to different integer samples-per-symbol configurations used during signal generation.

6.5 SNR Configuration

The final parameter dataset uses seven SNR conditions:

\[

$\{-10, -5, 0, 5, 10, 15, 20\} \text{ dB}$

\]

This range provides both low-quality and relatively clean signal conditions.

The SNR variation is important because modulation classification and parameter estimation become more challenging as the noise level increases.

6.6 Carrier Frequency Offset Configuration

Carrier Frequency Offset is introduced during signal generation to simulate transmitter-receiver frequency mismatch.

The generated CFO values are randomly selected within approximately:

$$\begin{aligned} & \setminus[\\ & [-3000, 3000] \setminus \text{Hz} \\ & \setminus] \end{aligned}$$

The end-to-end evaluation also analyzes the CFO estimation error across different sections of this range.

The CFO ranges used for error analysis are:

Range	CFO
Range 1	-3000 to -2000 Hz
Range 2	-2000 to -1000 Hz
Range 3	-1000 to 0 Hz
Range 4	0 to 1000 Hz
Range 5	1000 to 2000 Hz
Range 6	2000 to 3000 Hz

6.7 Dataset Generation

The project uses a controlled synthetic signal-generation pipeline.

For every generated sample, the following information is known:

- Modulation class
- SNR
- Sample rate
- Symbol rate
- Carrier Frequency Offset
- Signal samples

This ground-truth information makes supervised learning possible and allows prediction errors to be measured quantitatively.

Random seeds are used during dataset generation to improve reproducibility.

6.8 SNR Dataset

An SNR-focused dataset was generated during development containing:

\[

14,000

\]

signals.

The dataset configuration included:

- 8 modulation classes
- 7 SNR levels
- 250 signals per modulation/SNR combination
- 1024 samples per signal

The SNR levels were:

\[

-10,-5,0,5,10,15,20\ dB

\]

A common train/test split was created from this dataset.

6.9 Parameter Dataset

A larger parameter-estimation dataset was created containing:

\[

33,600

\]

signals.

The dataset combines:

- 8 modulation classes
- 7 SNR levels
- 6 symbol-rate conditions
- Multiple CFO values
- 1024 samples per signal

The common parameter dataset was divided into:

Split	Samples
Training	23,520
Testing	10,080
Total	33,600

The same common data configuration was used to train and evaluate the parameter-estimation models.

6.10 Parameter Dataset V2

A second version of the parameter dataset was generated using physically meaningful integer samples-per-symbol configurations.

The corresponding symbol rates were:

Samples per Symbol	Symbol Rate
20	50,000 Hz
13	76,923.08 Hz
10	100,000 Hz
8	125,000 Hz
7	142,857.14 Hz
5	200,000 Hz

The V2 dataset also contains:

\[
33,600
\]

signals and uses the same:

- 8 modulation classes

- 7 SNR levels
- 6 symbol-rate conditions

The resulting common split contains:

- 23,520 training signals
- 10,080 testing signals

This dataset configuration was used for the final parameter-estimation models.

6.11 End-to-End Evaluation Dataset

A controlled end-to-end evaluation was performed independently across combinations of modulation, SNR, symbol rate, and repeated test samples.

The evaluation contains:

\[

$8 \times 7 \times 6 \times 5 = 1680$

\]

signals.

Therefore:

Parameter	Number of Conditions
Modulation classes	8
SNR levels	7
Symbol-rate classes	6
Repetitions	5
Total test signals	**1,680**

This evaluation provides a common basis for assessing the complete integrated system.

6.12 Training and Testing Strategy

The project separates model-development data from final end-to-end evaluation.

Training data is used to learn model parameters.

Testing data is used to measure performance on signals that were not used for model fitting.

For the parameter models, the common split consists of:

\[

23,520

\]

training samples and:

\[

10,080

\]

test samples.

The final integrated evaluation contains 1,680 controlled signals.

This separation reduces the possibility of evaluating the final system using the same samples used for training.

6.13 Software Environment

The project was implemented using Python.

The main software components include:

Component	Technology
Programming Language	Python
Deep Learning	PyTorch
Machine Learning	Scikit-learn
Numerical Computing	NumPy
Signal Processing	SciPy
Data Processing	Pandas
Model Serialization	Joblib
Visualization	Matplotlib
Dashboard	Streamlit
Testing	Pytest

The project uses a Python virtual environment to isolate its dependencies.

6.14 Hardware and Execution Environment

The models were trained and evaluated using a CPU-based Python environment.

The available PyTorch installation did not use CUDA during the project execution.

Therefore, model training and inference were performed using CPU computation.

The system was designed to remain executable without requiring a dedicated GPU.

6.15 CNN Training Configuration

The final CNN training configuration includes:

Parameter	Configuration
Input	2×1024 I/Q samples
Optimizer	Adam
Learning Rate	0.001
Batch Size	64
Loss Function	Cross-Entropy Loss
Output Classes	8
Architecture	1D CNN

The trained final CNN is stored as:

```
models/cnn_common.pth
```

6.16 SNR Model Configuration

SNR estimation is implemented using a Random Forest regression model.

The final model is stored as:

```
models/snr_parameter_rf.joblib
```

The model uses a 15-dimensional feature representation.

The performance is evaluated using:

- MAE
- RMSE
- $\sqrt{R^2}$

6.17 CFO Model Configuration

CFO estimation uses a Random Forest regression model based on 30 extracted signal features.

The final model is stored as:

```
models/cfo_parameter_rf.joblib
```

Evaluation metrics include:

- MAE
- RMSE
- (R^2)
- Correlation
- CFO-range-specific MAE

6.18 Symbol-Rate Model Configuration

Symbol-rate classification uses a Random Forest classifier based on 40 extracted features.

The model predicts one of six supported symbol-rate classes.

The final model is stored as:

```
models/symbol_rate_classifier.joblib
```

The evaluation uses:

- Classification accuracy
- MAE
- RMSE
- (R^2)
- Correlation

6.19 Evaluation Metrics

Classification and regression tasks require different evaluation measures.

Classification

For modulation and symbol-rate classification:

[

Accuracy=

$\frac{\text{Correct Predictions}}{\text{Total Predictions}}$

]

Confusion matrices are also generated to examine class-level performance.

Regression

For SNR and CFO estimation, the primary metric is Mean Absolute Error:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

RMSE is also calculated:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

The (R^2) coefficient is used to measure the proportion of target variance explained by the model.

6.20 Reproducibility

The signal-generation pipeline uses explicit random seeds.

This allows the same experimental conditions to be reproduced during testing.

The project also stores trained models and evaluation results so that the final dashboard can use the same trained models as the reported experiments.

The repository contains the source code, model files, result files, and configuration required to reproduce the implemented system within the documented environment.

6.21 Experimental Setup Summary

The final experimental setup can be summarized as follows:

Item	Configuration
Modulation Classes	8
Signal Length	1024 samples
Sample Rate	1 MHz

SNR Levels	-10 to 20 dB
Number of SNR Levels	7
Symbol Rates	6
CFO Range	Approximately ± 3000 Hz
Final End-to-End Signals	1,680
Modulation Model	1D CNN
Parameter Models	Random Forest
Deep Learning Framework	PyTorch
ML Framework	Scikit-learn
Dashboard	Streamlit

The experimental setup provides a controlled environment for evaluating the complete AI-based signal analysis system under a range of wireless signal conditions.

Chapter 7 — System Implementation

7.1 Introduction

The proposed AI-based wireless signal analysis system was implemented using Python. The implementation follows a modular software architecture in which signal generation, preprocessing, feature extraction, model training, inference, visualization, and testing are separated into individual components.

The implementation consists of two main layers:

1. ****AI and signal-processing layer****
2. ****User-interface and visualization layer****

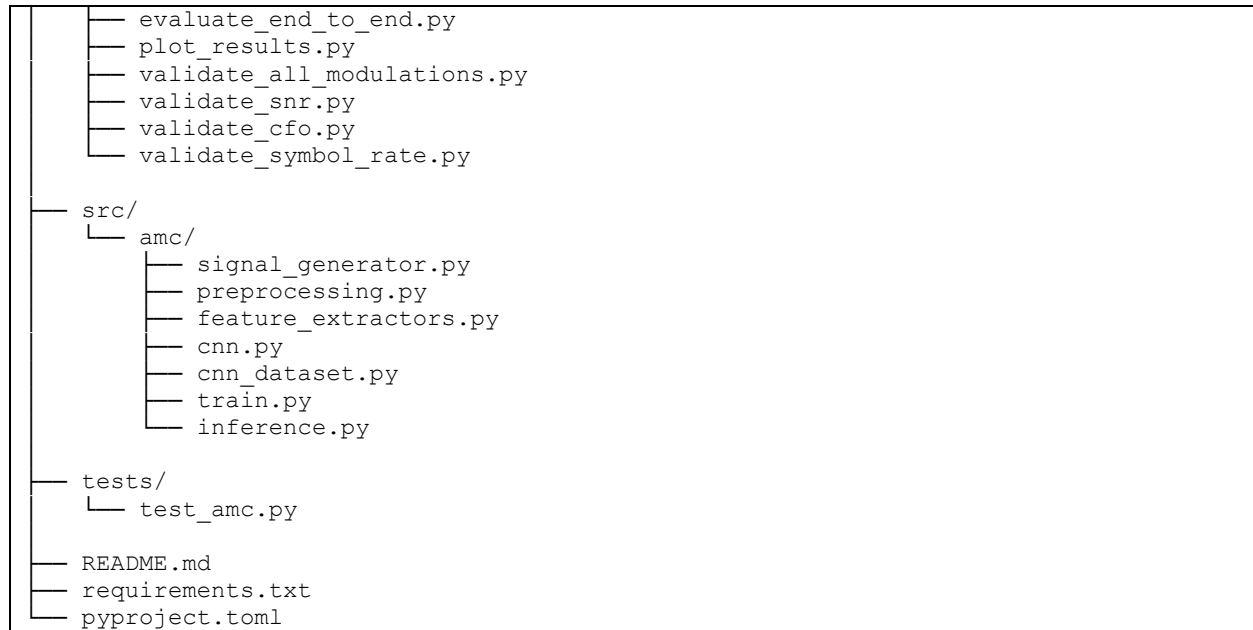
The AI layer performs modulation classification and signal parameter estimation, while the Streamlit layer provides an interactive interface for users.

7.2 Project Directory Structure

The project is organized into separate directories for source code, models, datasets, results, tests, and the dashboard.

The major structure is:

```
amc_project/
├── data/
│   ├── demo.npz
│   ├── snr_dataset.npz
│   ├── common_split.npz
│   ├── parameter_dataset.npz
│   ├── parameter_common_split.npz
│   ├── parameter_dataset_v2.npz
│   └── parameter_v2_common_split.npz
├── models/
│   ├── cnn_common.pth
│   ├── snr_parameter_rf.joblib
│   ├── cfo_parameter_rf.joblib
│   └── symbol_rate_classifier.joblib
├── results/
│   ├── end_to_end_modulation_confusion_matrix.png
│   ├── end_to_end_modulation_accuracy_by_class.png
│   ├── end_to_end_snr_mae_vs_snr.png
│   ├── end_to_end_cfo_mae_by_range.png
│   ├── end_to_end_symbol_rate_confusion_matrix.png
│   └── final_system_performance.png
├── dashboard/
│   └── app.py
└── scripts/
```



This organization makes the project modular and easier to maintain.

7.3 Signal Generator Implementation

The signal-generation functionality is implemented in:

```
src/amc/signal_generator.py
```

The module provides functions for generating complex I/Q signals.

The supported modulation classes are:

```
BPSK
QPSK
8PSK
16QAM
64QAM
GFSK
AM
FM
```

The main signal-generation function accepts parameters including:

- Modulation label
- Number of samples
- SNR

- Sample rate
- Symbol rate
- CFO
- Random seed

The generated signal is returned as a complex-valued NumPy array together with its associated parameter metadata.

7.4 Digital Modulation Implementation

The signal generator creates symbols according to the selected modulation scheme.

BPSK

Two phase states are represented using real-valued symbols:

$$s_k \in \{-1, +1\}$$

QPSK and 8PSK

PSK symbols are generated using equally spaced phase states:

$$s_k = e^{j2\pi m/M}$$

where M is the number of phase states.

16QAM and 64QAM

QAM signals use combinations of discrete real and imaginary amplitude levels.

The resulting complex symbols are converted into a time-domain sequence using the selected samples-per-symbol value.

7.5 Analog Modulation Implementation

The signal generator also implements AM and FM.

For AM, the generated baseband component is used to construct an amplitude-varying carrier representation.

For FM, the signal phase is varied according to the input waveform.

This allows the CNN to learn differences between digital and analog modulation characteristics.

7.6 GFSK Implementation

Gaussian Frequency Shift Keying is represented using a frequency-varying complex signal.

The implementation uses the generated signal to create a continuously varying phase representation.

This produces a signal structure that is distinct from PSK and QAM signals.

7.7 Noise and CFO Implementation

After the base signal is generated, carrier-frequency offset and complex Gaussian noise are introduced.

CFO is applied using:

$$\begin{aligned} & \backslash [\\ & x_{\text{CFO}}[n] \\ & = \\ & x[n]e^{j2\pi f_{\text{CFO}}t_n} \\ & \backslash] \end{aligned}$$

Noise is then added to produce the required SNR condition:

$$\begin{aligned} & \backslash [\\ & x_{\text{received}}[n] = x_{\text{CFO}}[n] + w[n] \\ & \backslash] \end{aligned}$$

Finally, the signal is normalized to maintain a consistent signal representation.

7.8 Preprocessing Implementation

Preprocessing is implemented in:

<code>src/amc/preprocessing.py</code>

The main operations include:

1. Conversion to complex-valued representation.
2. Mean removal.
3. Power-based normalization.
4. Extraction of basic signal characteristics.
5. Conversion of the signal into the representation required by the models.

The normalization process reduces the effect of absolute signal amplitude on model predictions.

7.9 CNN Dataset Implementation

The CNN dataset handling is implemented in:

```
src/amc/cnn_dataset.py
```

The dataset loader converts each complex signal into two real-valued channels.

The resulting tensor has the structure:

```
Channel 0 → I component  
Channel 1 → Q component
```

For a 1024-sample signal:

```
Input shape = (2, 1024)
```

For a batch of signals:

```
Input shape = (Batch, 2, 1024)
```

Per-sample normalization is applied before the signal is passed to the CNN.

7.10 CNN Implementation

The CNN architecture is implemented in:

```
src/amc/cnn.py
```

The model is named `AMCNet`.

The network contains four convolutional stages followed by adaptive pooling and fully connected layers.

The main architecture is:

```
Input
↓
Conv1D: 2 → 32
↓
BatchNorm + ReLU + MaxPool
↓
Conv1D: 32 → 64
↓
BatchNorm + ReLU + MaxPool
↓
Conv1D: 64 → 128
↓
BatchNorm + ReLU + MaxPool
↓
Conv1D: 128 → 128
↓
BatchNorm + ReLU
↓
Adaptive Average Pooling
↓
Linear: 128 → 64
↓
Dropout
↓
Linear: 64 → 8
↓
Modulation Prediction
```

The final output consists of eight class scores.

7.11 Model Training

The model-training functionality is implemented through the project's training scripts.

The CNN uses:

- Adam optimizer
- Learning rate: 0.001
- Batch size: 64
- Cross-entropy loss

The model learns from labeled I/Q signals.

During training, the CNN parameters are updated to reduce the classification loss.

The final trained modulation model is saved as:

```
models/cnn_common.pth
```

7.12 Feature Extractor Implementation

Parameter-estimation features are implemented in:

```
src/amc/feature_extractors.py
```

Three feature extraction functions are provided:

```
snr_features()  
cfo_features()  
symbol_rate_features()
```

Their output dimensions are:

Feature Extractor	Output
SNR	15 features
CFO	30 features
Symbol Rate	40 features

The features include statistical, amplitude, phase, spectral, and signal-structure information.

7.13 SNR Model Implementation

The SNR estimator uses a Random Forest regression model.

The model receives the 15-dimensional SNR feature vector.

The model learns the relationship:

$$\begin{bmatrix} X_{\text{SNR}} \end{bmatrix} \rightarrow \text{SNR}$$

The trained model is stored as:

```
models/snr_parameter_rf.joblib
```

During inference, the feature extractor processes the input signal and the Random Forest model returns the estimated SNR in dB.

7.14 CFO Model Implementation

The CFO estimator uses a Random Forest regression model with 30 input features.

The processing pipeline is:

```
I/Q Signal
↓
CFO Feature Extraction
↓
30-Dimensional Feature Vector
↓
Random Forest Regression
↓
Estimated CFO
```

The trained model is stored as:

```
models/cfo_parameter_rf.joblib
```

The output is expressed in Hertz.

7.15 Symbol-Rate Model Implementation

The symbol-rate estimator is implemented as a Random Forest classifier.

The input consists of 40 extracted signal features.

The supported output classes are:

```
50 kHz
76.923 kHz
100 kHz
125 kHz
142.857 kHz
200 kHz
```

The trained model is stored as:

```
models/symbol_rate_classifier.joblib
```

The model predicts the most probable supported symbol-rate class.

7.16 Inference Module

The complete model-inference pipeline is implemented in:

```
src/amc/inference.py
```

The inference module loads all final trained models:

```
cnn_common.pth  
snr_parameter_rf.joblib  
cfo_parameter_rf.joblib  
symbol_rate_classifier.joblib
```

The module then processes an input signal and returns a combined analysis result.

The output includes:

- Modulation
- Modulation confidence
- Modulation probabilities
- SNR
- CFO
- Symbol rate
- Sample rate
- Number of samples

7.17 Input Validation

The dashboard implements validation for uploaded signal files.

Supported formats include:

```
.npy  
.npz  
.csv
```

The input-processing system checks:

- Whether the file can be loaded.
- Whether signal samples are present.
- Whether values are finite.
- Whether the signal format is supported.

- Whether the signal can be converted into the required representation.

Invalid inputs are rejected before model inference.

This prevents malformed data from being passed to the AI models.

7.18 Streamlit Dashboard Implementation

The user interface is implemented in:

```
dashboard/app.py
```

The dashboard is built using Streamlit.

The main dashboard workflow is:

```
Select Input Mode
      ↓
Generate / Upload Signal
      ↓
Validate Signal
      ↓
Analyze Signal
      ↓
Load AI Models
      ↓
Generate Predictions
      ↓
Display Results
      ↓
Visualize Signal
      ↓
Export Analysis Report
```

7.19 Dashboard Input Modes

The dashboard provides two input modes.

Mode 1 — Generate Synthetic Signal

Users can select signal parameters directly from the interface.

Available controls include:

- Modulation
- SNR
- CFO
- Symbol rate

- Number of samples
- Random seed

The generated signal is immediately available for AI analysis.

Mode 2 — Upload I/Q Signal

Users can upload supported I/Q files.

The dashboard reads and validates the file before processing it.

This provides an interface for testing externally prepared signals.

7.20 Dashboard Analysis Output

After analysis, the dashboard displays an **AI Signal Analysis** section.

The main predictions include:

Predicted Modulation
Modulation Confidence
Estimated SNR
Estimated CFO
Estimated Symbol Rate

For generated signals, the dashboard can also compare:

Input Parameter
vs.
AI Estimated Parameter

and calculate the corresponding estimation errors.

7.21 Signal Visualization

The dashboard provides several signal visualizations.

I/Q Waveform

Displays the time-domain In-phase and Quadrature components.

Constellation

Displays the relationship between the I and Q components in the complex plane.

Frequency Spectrum

Displays the frequency-domain characteristics of the signal.

Amplitude and Phase

Displays the amplitude and phase behavior of the received signal.

These visualizations help users interpret the signal characteristics alongside the AI predictions.

7.22 Performance Visualization

The dashboard also displays final model-performance results.

The reported final system metrics are:

Metric	Result
Modulation Accuracy	75.24%
SNR MAE	0.467 dB
CFO MAE	688.56 Hz
Symbol-Rate Accuracy	70.95%

The dashboard also provides:

- Modulation accuracy versus SNR
- Modulation accuracy by class
- Modulation confusion matrix
- Symbol-rate confusion matrix

7.23 Testing Implementation

Automated tests are implemented in:

```
tests/test_amc.py
```

The test suite verifies important functionality of the project.

The final test execution completed successfully:

```
3 passed
```

This confirms that the implemented core functionality covered by the automated test suite is functioning as expected.

7.24 Validation Scripts

Additional validation scripts were developed to test individual components.

The project includes validation for:

```
validate_all_modulations.py  
validate_snr.py  
validate_cfo.py  
validate_symbol_rate.py
```

These scripts provide controlled spot checks of the individual models.

For example, the modulation validation checked all eight supported modulation classes.

The end-to-end evaluation is used as the primary quantitative evaluation of the integrated system.

7.25 End-to-End Evaluation Implementation

The final integrated evaluation is performed using:

```
scripts/evaluate_end_to_end.py
```

The evaluation tests:

\[

1680

\]

controlled signals.

The evaluation combines:

- 8 modulation classes
- 7 SNR levels
- 6 symbol-rate conditions
- 5 repetitions

The results are saved in:

```
models/end_to_end_results.csv
```

The resulting metrics are then used to generate the final performance visualizations.

7.26 Result Visualization Implementation

The final visualization script is:

```
scripts/plot_results.py
```

It generates the final result figures used for analysis.

The generated figures include:

```
end_to_end_modulation_confusion_matrix.png  
end_to_end_modulation_accuracy_by_class.png  
end_to_end_snr_mae_vs_snr.png  
end_to_end_cfo_mae_by_range.png  
end_to_end_symbol_rate_confusion_matrix.png  
final_system_performance.png
```

These figures provide visual evidence of the system's performance.

7.27 Model and Result Storage

The final trained models are stored separately from the source code.

The final model directory contains:

```
models/  
├── cnn_common.pth  
├── snr_parameter_rf.joblib  
├── cfo_parameter_rf.joblib  
└── symbol_rate_classifier.joblib
```

The model files are version-controlled using Git Large File Storage (Git LFS) because of their size.

The project repository also contains the final evaluation results and visualization files required to document the system.

7.28 Version Control and Repository

The project is maintained using Git.

The final repository is publicly available on GitHub.

The source repository contains:

- Source code
- Dashboard
- Testing code

- Documentation
- Model files
- Final evaluation results
- Visualization scripts

Git LFS is used for the large trained model files.

The final implementation was pushed to the `main` branch and verified with a clean working tree.

7.29 Implementation Summary

The implementation converts the proposed architecture into a complete executable system.

The software is divided into independent modules for signal generation, preprocessing, feature extraction, deep learning, machine learning, inference, visualization, and testing.

The final implementation therefore provides both the underlying AI-based signal-analysis functionality and a user-friendly dashboard for interacting with the system.

Chapter 8 — Results and Discussion

8.1 Introduction

The developed system was evaluated to measure its ability to classify modulation schemes and estimate important signal parameters. The evaluation was performed using controlled synthetic I/Q signals with known ground-truth modulation, SNR, CFO, and symbol-rate values.

The final end-to-end evaluation consisted of **1,680 signals**, covering eight modulation classes, seven SNR levels, six symbol-rate conditions, and five repeated test signals for each combination.

The evaluation considered four primary tasks:

1. Modulation classification
2. SNR estimation
3. CFO estimation
4. Symbol-rate classification

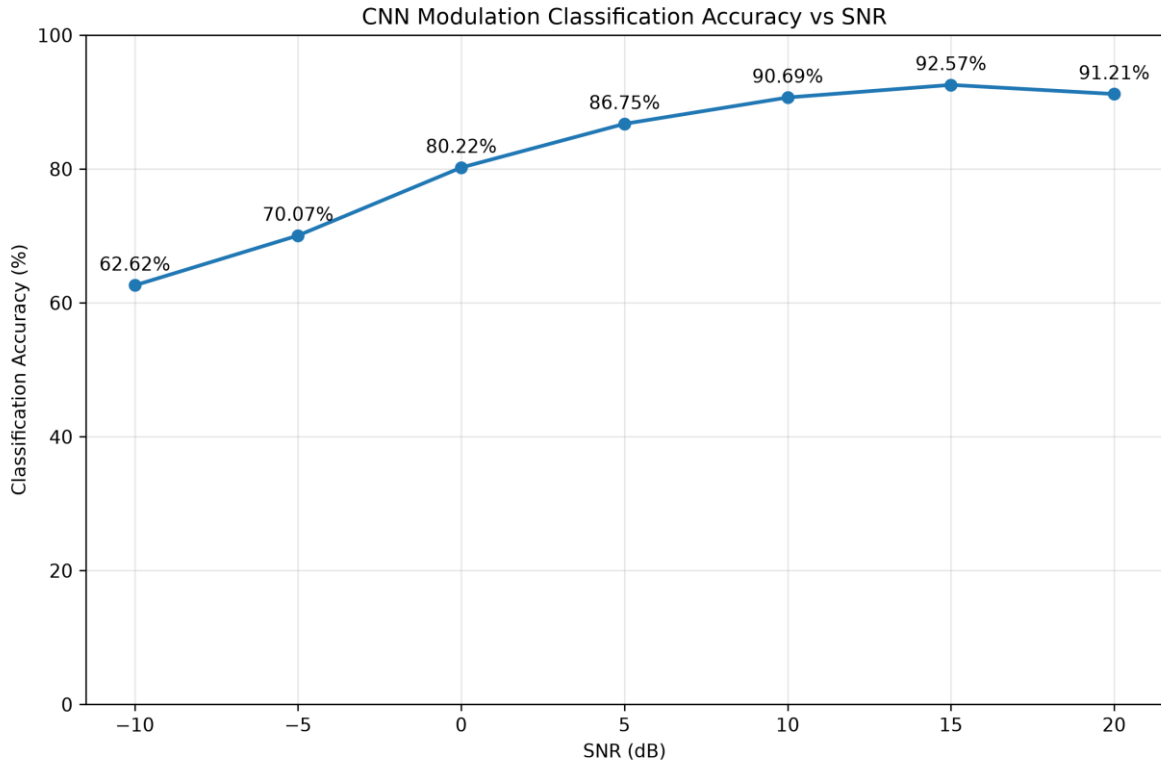
8.2 Overall System Performance

The final end-to-end performance is summarized below.

Task	Metric	Result
Modulation Classification	Accuracy	**75.24%**
SNR Estimation	MAE	**0.467 dB**
SNR Estimation	RMSE	**1.069 dB**
CFO Estimation	MAE	**688.56 Hz**
CFO Estimation	RMSE	**1034.11 Hz**
Symbol-Rate Classification	Accuracy	**70.95%**
End-to-End Evaluation	Test Signals	**1,680**

The results demonstrate that the integrated system is capable of performing all four intended analysis tasks.

8.3 Modulation Classification Performance



The overall modulation-classification accuracy obtained during the final end-to-end evaluation was:

$$\boxed{75.24\%}$$

The model was evaluated across all eight supported modulation classes.

The class-wise results were:

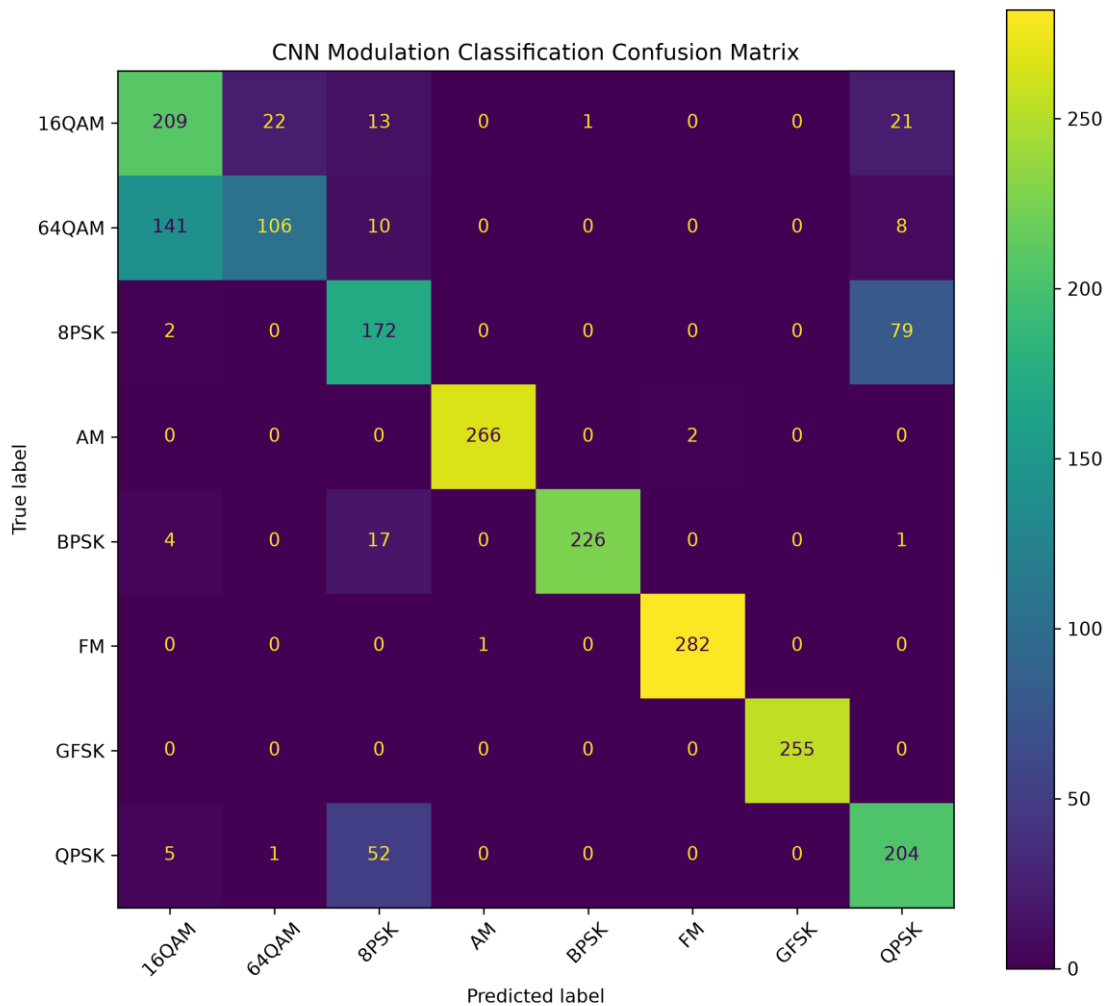
Modulation	Accuracy
BPSK	93.81%
QPSK	68.57%
8PSK	60.48%
16QAM	56.67%
64QAM	33.33%
GFSK	97.62%
AM	96.19%
FM	95.24%
Overall	**75.24%**

The results show that the classifier performs differently across modulation classes.

BPSK, GFSK, AM, and FM achieve relatively high classification accuracy in the evaluated dataset. In contrast, 64QAM has lower accuracy, followed by 16QAM.

This difference can be attributed to the greater similarity between some higher-order digital modulation patterns under noisy conditions. In particular, distinguishing 16QAM and 64QAM can be more challenging because both use amplitude and phase variations.

8.4 Modulation Confusion Matrix



A confusion matrix was generated to examine the classification behavior across the eight modulation classes.

The final confusion matrix is stored as:

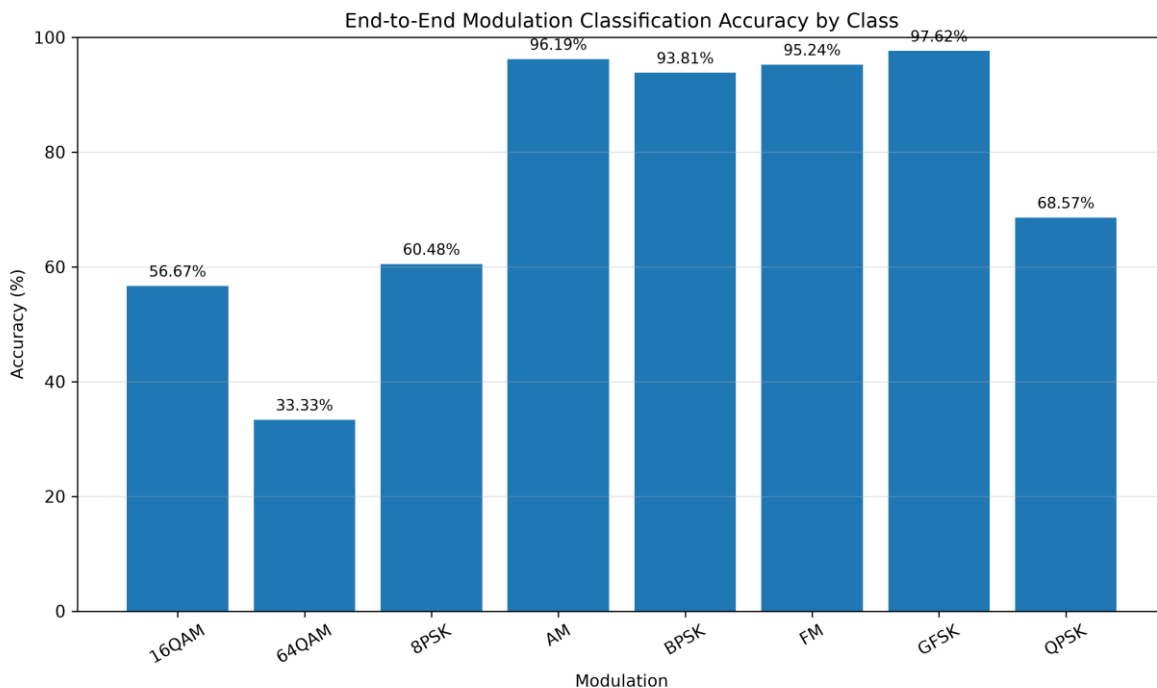
```
results/end_to_end_modulation_confusion_matrix.png
```

The confusion matrix provides the number of correct and incorrect predictions for every combination of true and predicted modulation class.

The matrix is useful for identifying which modulation classes are confused with each other.

The class-wise results indicate that the model distinguishes the analog modulation classes and GFSK relatively well, while higher-order QAM classes remain more difficult.

8.5 Modulation Accuracy by Class



A class-wise accuracy visualization was generated:

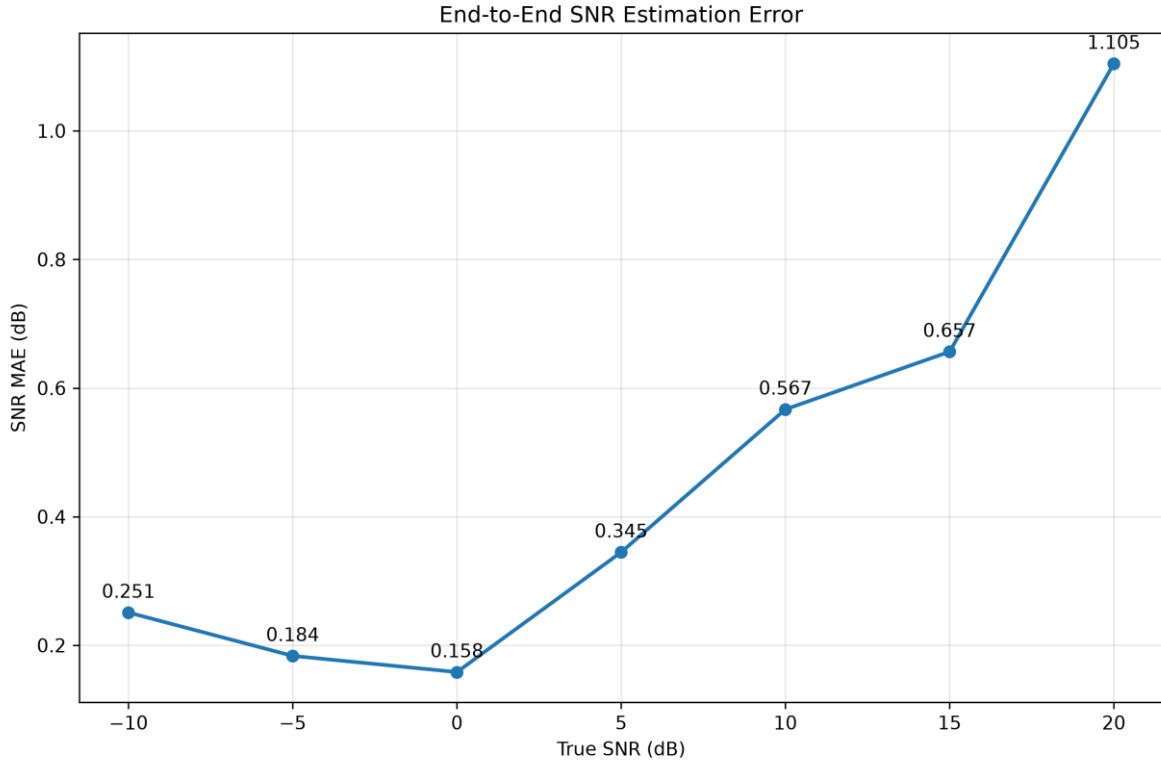
`results/end to end modulation accuracy by class.png`

The visualization shows the variation in classification accuracy among the eight classes.

The results demonstrate that overall accuracy alone does not completely describe model behavior. Some modulation classes have substantially higher classification accuracy than others.

Therefore, class-wise analysis is important when evaluating a multi-class AMC system.

8.6 Modulation Performance versus SNR



The system was evaluated at seven SNR levels:

[
-10, -5, 0, 5, 10, 15, 20 dB
]

The modulation-classification accuracy across SNR conditions was:

SNR	Modulation Accuracy
-10 dB	56.61%
-5 dB	66.84%
0 dB	82.02%
5 dB	88.52%
10 dB	91.07%
15 dB	89.78%
20 dB	90.10%

The results show a clear improvement as the signal quality increases from very low SNR conditions.

At -10 dB, the signal contains substantial noise, making modulation-specific patterns more difficult for the CNN to identify.

At 0 dB and above, classification performance improves substantially.

The highest measured accuracy in this evaluation occurs at 10 dB SNR.

At higher SNR values, performance remains approximately stable rather than increasing indefinitely.

The corresponding visualization is:

`results/cnn_accuracy_vs_snr.png`

and the final end-to-end SNR analysis is represented through the project's result plots.

8.7 SNR Estimation Performance

The SNR estimator achieved a final end-to-end Mean Absolute Error of:

$\boxed{0.467 \text{ dB}}$

and an RMSE of:

$\boxed{1.069 \text{ dB}}$

The SNR estimation error for each true SNR condition was:

True SNR	MAE
-10 dB	0.251 dB
-5 dB	0.184 dB
0 dB	0.158 dB
5 dB	0.345 dB
10 dB	0.567 dB
15 dB	0.657 dB
20 dB	1.105 dB

The results show relatively small errors across the evaluated SNR range.

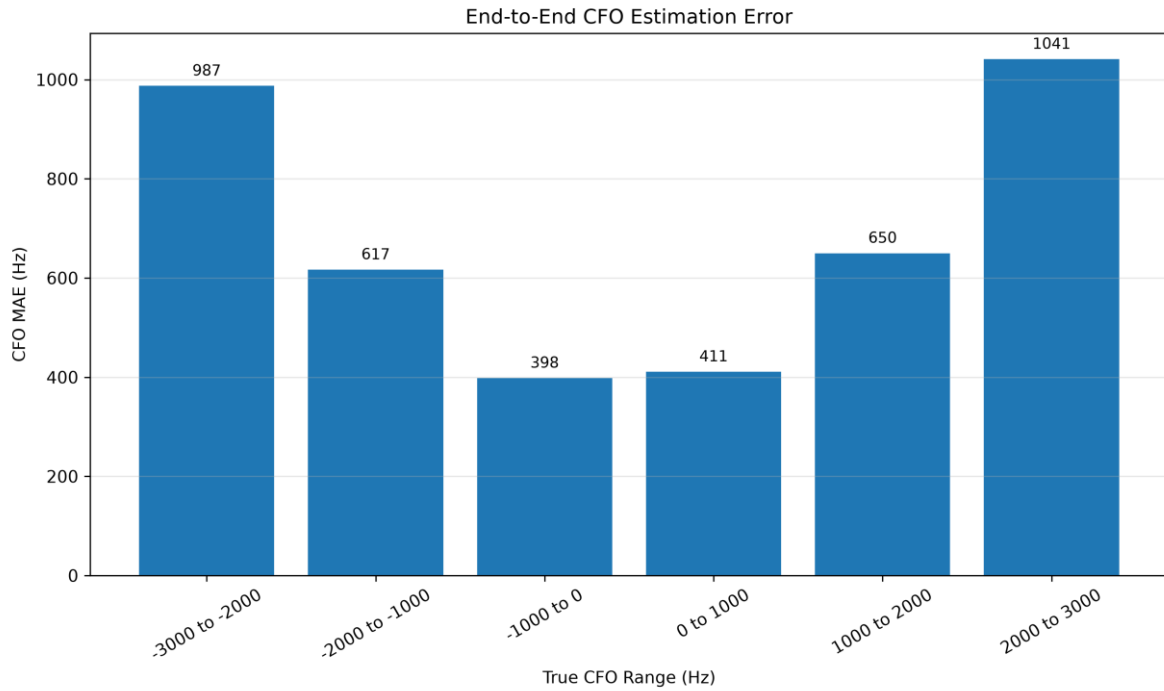
The smallest measured error occurs around 0 dB, while the largest error occurs at 20 dB.

This indicates that the current model estimates moderate SNR conditions more accurately than some high-SNR conditions in the evaluated dataset.

The final visualization is:

results/end_to_end_snr_mae_vs_snr.png

8.8 CFO Estimation Performance



The CFO estimator achieved:

[
 $\boxed{\text{MAE}=688.56\text{ Hz}}$
]

with:

[
 $\boxed{\text{RMSE}=1034.11\text{ Hz}}$
]

The CFO estimation error was also analyzed according to the CFO range.

CFO Range	MAE
-3000 to -2000 Hz	987.22 Hz
-2000 to -1000 Hz	616.69 Hz

-1000 to 0 Hz	398.05 Hz
0 to 1000 Hz	411.03 Hz
1000 to 2000 Hz	649.84 Hz
2000 to 3000 Hz	1041.26 Hz

The results show that the estimator performs more accurately near the center of the evaluated CFO range.

The error increases toward the negative and positive boundaries.

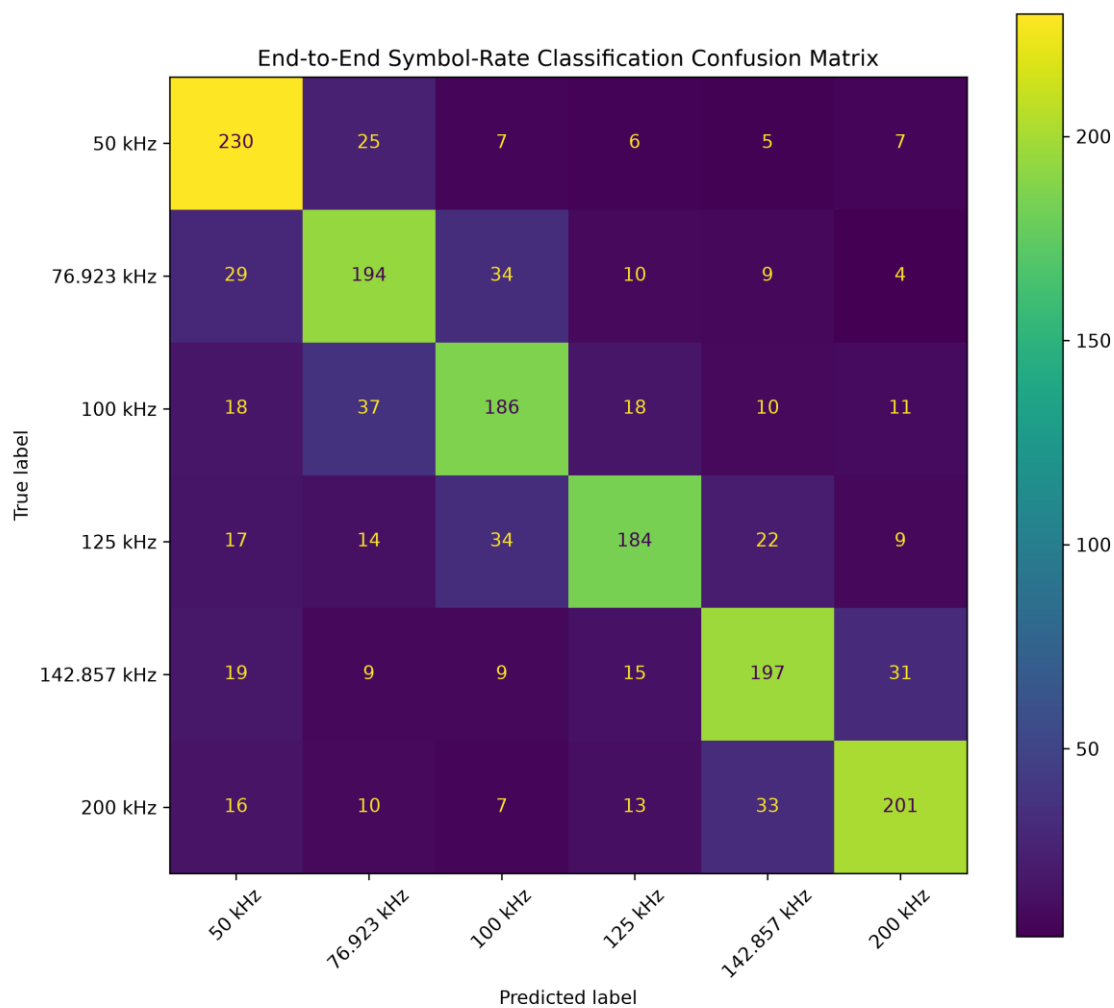
This behavior indicates that the current feature-based model has difficulty maintaining the same estimation precision near the extremes of the supported CFO range.

The corresponding visualization is:

`results/end_to_end_cfo_mae_by_range.png`

CFO is therefore the most challenging parameter-estimation task in the current implementation.

8.9 Symbol-Rate Classification Performance



The symbol-rate model classifies the input signal into one of six supported symbol-rate classes.

The final end-to-end accuracy was:

$$\boxed{70.95\%}$$

The supported rates are:

- 50 kHz
- 76.923 kHz
- 100 kHz

- 125 kHz
- 142.857 kHz
- 200 kHz

The corresponding confusion matrix is stored as:

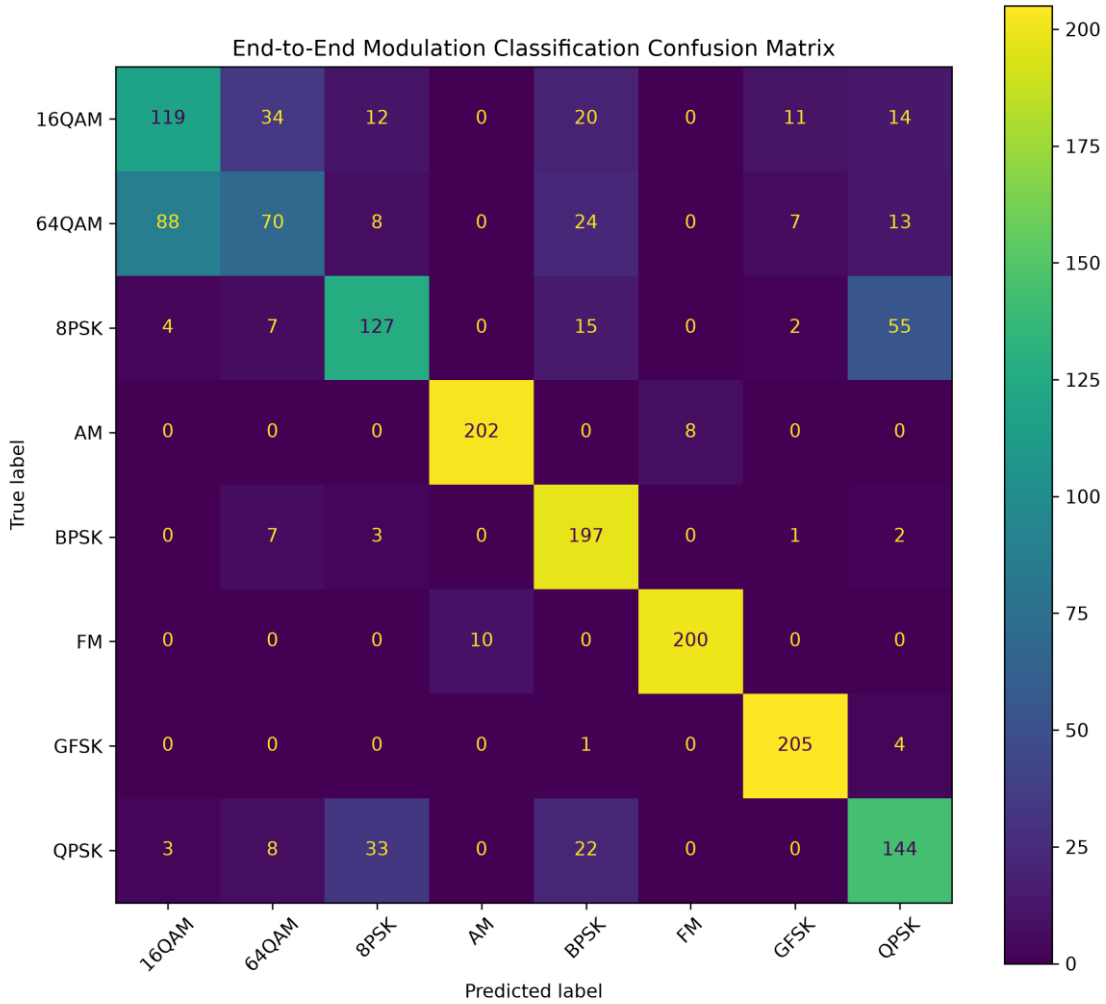
<code>results/end_to_end_symbol_rate_confusion_matrix.png</code>
--

The confusion matrix shows the distribution of correct and incorrect symbol-rate predictions.

Because several supported rates are relatively close to one another, distinguishing them from a short noisy signal sequence is challenging.

The model therefore provides classification among the predefined rates rather than unrestricted continuous symbol-rate estimation.

8.10 Comparison of Parameter Estimation Tasks



The three parameter-estimation components show different levels of performance.

Parameter	Evaluation Method	Result
SNR	Regression	MAE = 0.467 dB
CFO	Regression	MAE = 688.56 Hz
Symbol Rate	Classification	Accuracy = 70.95%

SNR estimation provides the smallest normalized error relative to its operating range.

CFO estimation has a larger error, particularly toward the boundaries of the evaluated range.

Symbol-rate classification provides moderate performance across the six predefined classes.

8.11 Final System Performance Visualization

AI Wireless Signal Analyzer - Final End-to-End Performance

Modulation Accuracy : 75.24%
SNR MAE : 0.467 dB
CFO MAE : 688.56 Hz
Symbol Rate Accuracy: 70.95%

A combined performance visualization was generated:

results/final_system_performance.png

This figure provides a consolidated view of the principal performance metrics.

The final system demonstrates that different components of the integrated architecture have different performance characteristics.

The modulation classifier provides multi-class classification, while the parameter-estimation models provide numerical or class-based estimates of signal properties.

8.12 Controlled Spot-Check Validation

In addition to the large end-to-end evaluation, individual validation scripts were used to perform controlled spot checks.

The modulation validation tested all eight supported modulation classes.

The observed predictions for the controlled validation samples were:

True Modulation	Predicted Modulation
BPSK	BPSK
QPSK	QPSK
8PSK	8PSK
16QAM	16QAM
64QAM	64QAM

GFSK	GFSK
AM	AM
FM	FM

This spot check confirms that each supported class could be correctly identified for the particular controlled samples tested.

However, these individual spot checks should not be interpreted as overall model accuracy. The 1,680-signal end-to-end evaluation remains the primary performance measurement.

8.13 Discussion of Results

The experimental results demonstrate the feasibility of combining deep learning and feature-based machine learning for wireless signal analysis.

The modulation classifier achieved 75.24% overall accuracy across the complete end-to-end evaluation. The performance varies substantially among modulation classes, with higher-order QAM classes being more difficult to classify.

The SNR estimator achieved an MAE of 0.467 dB, indicating comparatively accurate SNR estimation for the controlled signal conditions.

CFO estimation achieved an MAE of 688.56 Hz. The range-specific results indicate that estimation error increases near the limits of the evaluated CFO range.

The symbol-rate classifier achieved 70.95% accuracy across six predefined symbol-rate classes.

These results demonstrate that the system can perform multiple signal-analysis tasks within a single integrated platform while also revealing specific areas where further model development is required.

8.14 Limitations Identified from the Results

The experimental results identify several limitations.

1. Modulation-Class Imbalance in Performance

Although the overall modulation accuracy is 75.24%, the performance is not uniform across all classes.

The 64QAM class achieves substantially lower accuracy than several other classes.

2. CFO Estimation Error

CFO estimation has a relatively high absolute error compared with SNR estimation.

The error increases near the boundaries of the evaluated CFO range.

3. Restricted Symbol-Rate Classes

The symbol-rate model currently predicts only six predefined symbol rates.

It is therefore not a general-purpose continuous symbol-rate estimator.

4. Synthetic Dataset

The evaluation uses controlled synthetic signals.

Real-world signals can contain additional effects such as multipath propagation, fading, timing offsets, filtering, nonlinear distortion, interference, and hardware-specific impairments that are not fully represented in the current dataset.

5. Fixed Signal Length

The primary CNN inference pipeline uses 1024 samples.

Signals with substantially different characteristics may require additional preprocessing or segmentation.

8.15 Overall Assessment

The final experimental results show that the developed system successfully integrates multiple AI-based wireless signal-analysis functions.

The complete system can:

- Classify eight modulation schemes.
- Estimate SNR.
- Estimate CFO.
- Classify six supported symbol-rate conditions.
- Visualize signal characteristics.
- Present the results through an interactive dashboard.

The evaluation also provides quantitative evidence of the strengths and limitations of each component.

The results establish a baseline for future improvements using larger datasets, real-world RF recordings, more advanced neural architectures, and improved signal-parameter estimation techniques.

Chapter 9 — Streamlit Dashboard

9.1 Introduction

To provide an accessible interface for the developed AI-based wireless signal analysis system, an interactive dashboard was implemented using **Streamlit**.

The dashboard provides a graphical interface through which users can generate synthetic signals or upload I/Q signal files, execute AI-based analysis, inspect estimated signal parameters, visualize signal characteristics, and review model performance.

The dashboard acts as the user-facing layer of the complete system.

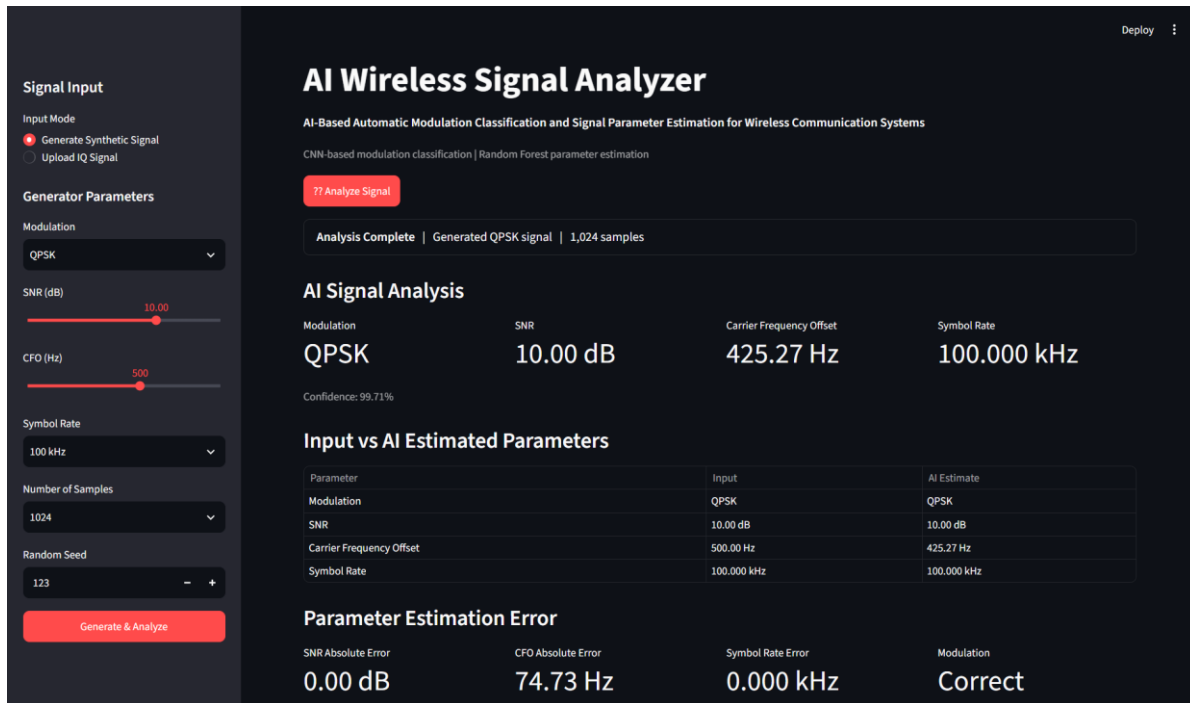
9.2 Dashboard Architecture

The dashboard follows the processing sequence:

```
User
↓
Input Selection
↓
Generate / Upload I/Q Signal
↓
Input Validation
↓
Signal Analysis
↓
AI Models
↓
Results
↓
Visualization
↓
Performance Analysis
↓
Report Export
```

The dashboard communicates with the trained models and signal-processing modules developed as part of the project.

9.3 Dashboard Launch



The dashboard is implemented in:

```
dashboard/app.py
```

It can be launched from the project environment using:

```
streamlit run dashboard/app.py
```

The application runs locally through the Streamlit server.

During development, the dashboard was successfully launched at:

```
http://localhost:8501
```

9.4 Input Modes

The dashboard provides two methods for supplying a signal.

9.4.1 Generate Synthetic Signal

The user can generate a controlled signal directly from the dashboard.

The available parameters include:

- Modulation

- SNR
- Carrier Frequency Offset
- Symbol Rate
- Number of Samples
- Random Seed

This mode is useful for testing the system when the ground-truth signal parameters are known.

9.4.2 Upload I/Q Signal

The dashboard also supports external signal files.

The supported file formats are:

.npy .npz .csv

The uploaded signal is processed and passed through the same analysis pipeline.

For generated signals, the dashboard can compare the known input parameters against the AI estimates.

9.5 Input Validation

Input validation is performed before model inference.

The dashboard checks:

- File format
- Whether signal data can be loaded
- Whether samples are present
- Whether signal values are finite
- Whether the data can be converted into the required representation

Invalid input is rejected rather than passed to the trained models.

For uploaded files, the dashboard also displays information such as:

- Number of loaded samples

- File format
- Sample-rate information where available

This improves the reliability of the user-facing application.

9.6 AI Signal Analysis

After an input signal is supplied, the user can execute the analysis operation.

The dashboard displays the primary AI results in an **AI Signal Analysis** section.

The output includes:

Result	Description
Modulation	Predicted modulation class
Confidence	Confidence of the predicted modulation
SNR	Estimated SNR in dB
CFO	Estimated carrier-frequency offset in Hz
Symbol Rate	Estimated supported symbol-rate class

For modulation classification, the system also provides the predicted probability distribution across the supported modulation classes.

9.7 Input versus AI Estimated Parameters

Input vs AI Estimated Parameters

Parameter	Input	AI Estimate
Modulation	QPSK	QPSK
SNR	10.00 dB	10.00 dB
Carrier Frequency Offset	500.00 Hz	425.27 Hz
Symbol Rate	100.000 kHz	100.000 kHz

Parameter Estimation Error

SNR Absolute Error

0.00 dB

CFO Absolute Error

74.73 Hz

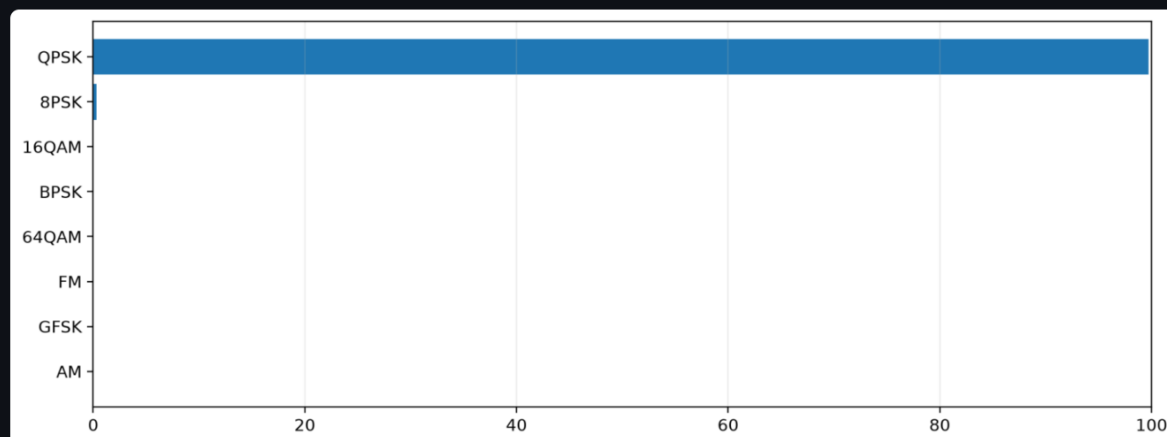
Symbol Rate Error

0.000 kHz

Modulation

Correct

Modulation Confidence Distribution



When a synthetic signal is generated, its parameters are known.

The dashboard can therefore compare:

Input Parameter
vs.
AI Estimated Parameter

The comparison includes parameters such as:

- SNR
- CFO
- Symbol rate

This allows the user to inspect the difference between the generated ground truth and the model prediction.

9.8 Parameter Estimation Error

For generated signals, the dashboard also presents the estimation error.

For a parameter y , the error can be calculated as:

$$\text{Error} = \hat{y} - y$$

where:

- y is the input or ground-truth value.
- $\hat{y}$ is the AI-estimated value.

This feature provides immediate feedback on the performance of the parameter-estimation models for the selected signal.

9.9 Modulation Confidence Distribution

The dashboard displays the modulation-class probability distribution.

The CNN produces probabilities for all eight supported modulation classes.

The predicted class corresponds to the class with the highest probability.

This visualization provides more information than the predicted class alone because it shows how the model distributes confidence across the possible modulation schemes.

9.10 I/Q Waveform Visualization

The dashboard displays the time-domain I/Q waveform.

The two components are shown separately:

$$I[n]$$

and:

$$Q[n]$$

This allows users to inspect the temporal behavior of the complex signal.

The waveform can help identify variations in amplitude, phase, and signal structure.

9.11 Constellation Visualization

The dashboard provides a constellation diagram based on the complex I/Q samples.

The constellation represents:

$$\begin{matrix} \backslash \\ I[n] \quad \text{versus} \quad Q[n] \\ \backslash \end{matrix}$$

The visualization is particularly useful for digital modulation schemes such as:

- BPSK
- QPSK
- 8PSK
- 16QAM
- 64QAM

The constellation can provide visual insight into the distribution of signal points and the effect of noise.

9.12 Frequency Spectrum

A frequency-spectrum visualization is provided to examine the frequency-domain characteristics of the signal.

The spectrum is calculated using the Fourier transform.

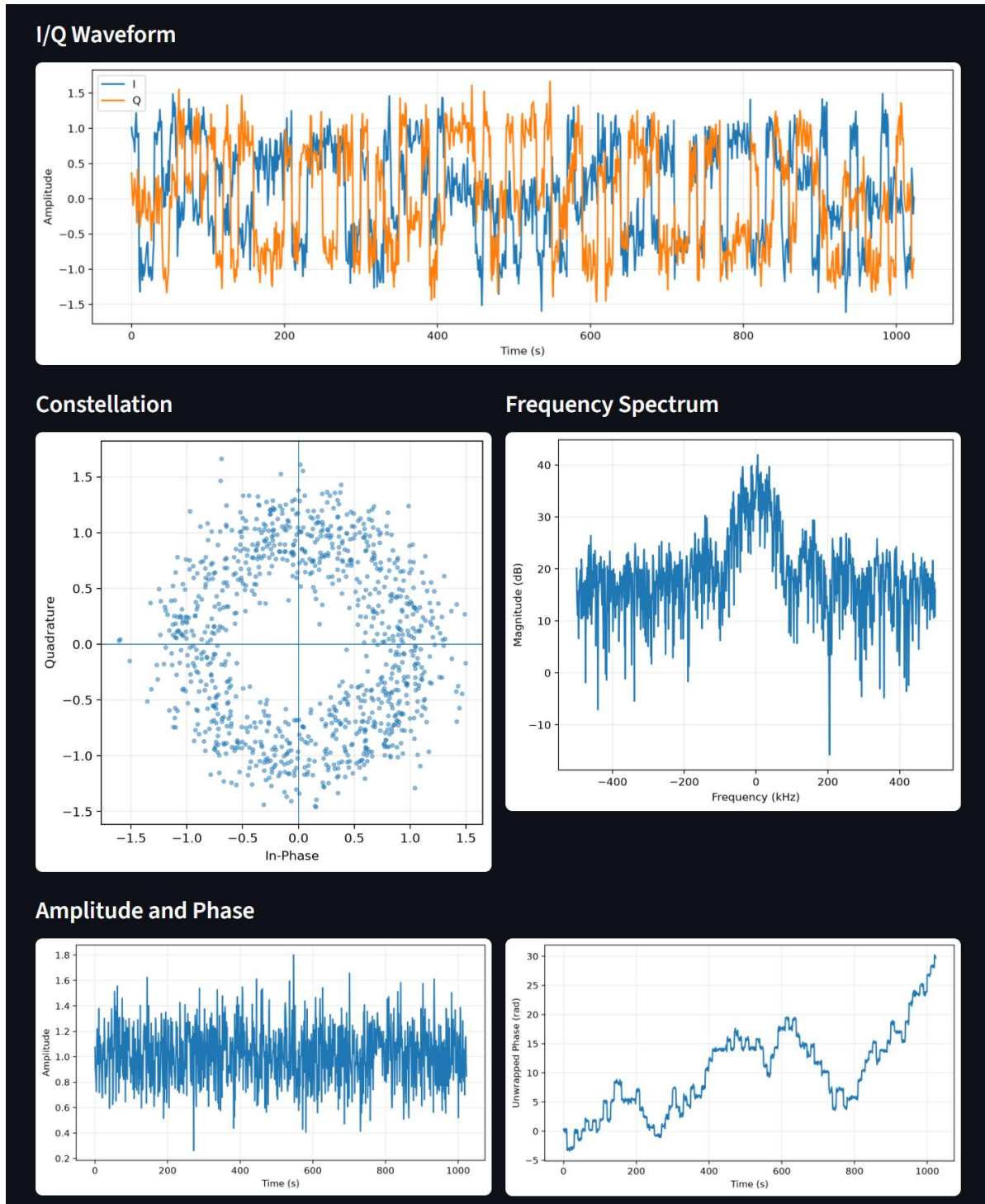
For a discrete signal $x[n]$, its frequency-domain representation is obtained using:

$$\begin{matrix} \backslash \\ X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \\ \backslash \end{matrix}$$

The magnitude of the resulting spectrum is displayed through the dashboard.

This visualization helps users inspect spectral characteristics and frequency-related behavior.

9.13 Amplitude and Phase Analysis



The dashboard also displays amplitude and phase information.

The signal amplitude is:

\[

$$A[n]=|x[n]|$$

\]

and its phase is:

\[

$$\phi[n]=\angle x[n]$$

\]

These representations provide additional information about the signal structure.

Amplitude characteristics can be useful for examining amplitude-based modulation, while phase characteristics are useful for analyzing phase- and frequency-related modulation behavior.

9.14 Technical Information

The dashboard provides a technical-information section containing information about the analyzed signal.

Depending on the input mode, this can include:

- Number of samples
- Sample rate
- Signal format
- Input source
- Analysis parameters

This provides context for interpreting the AI predictions and visualizations.

9.15 Model Performance Section

Technical Information

Modulation Classifier

1D Convolutional Neural Network

8 supported modulation classes

Parameter Estimation

Random Forest regression/classification

SNR + CFO + Symbol Rate

Signal

Sample Rate: 1.000 MHz

Samples: 1,024

Export Analysis Report

[Download Analysis Report](#)

Model Performance

Modulation Accuracy

75.24%

SNR MAE

0.47 dB

CFO MAE

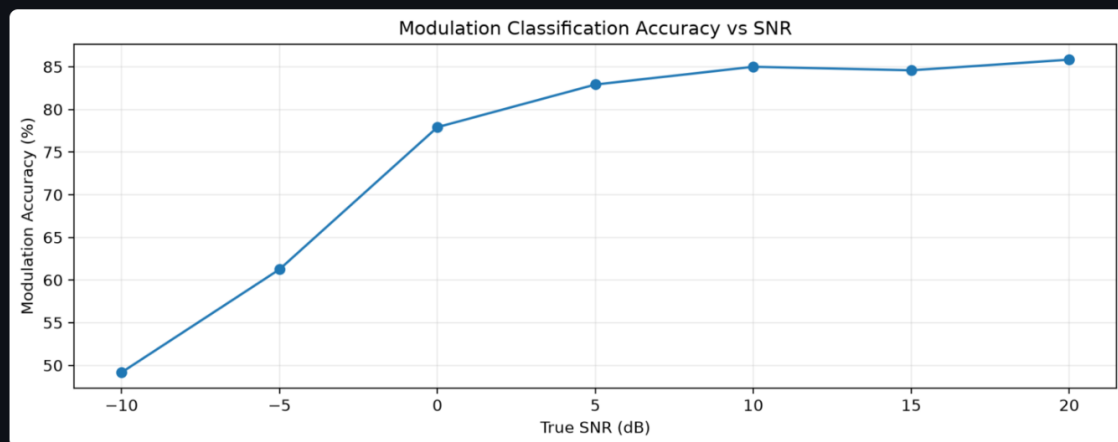
689 Hz

Symbol Rate Accuracy

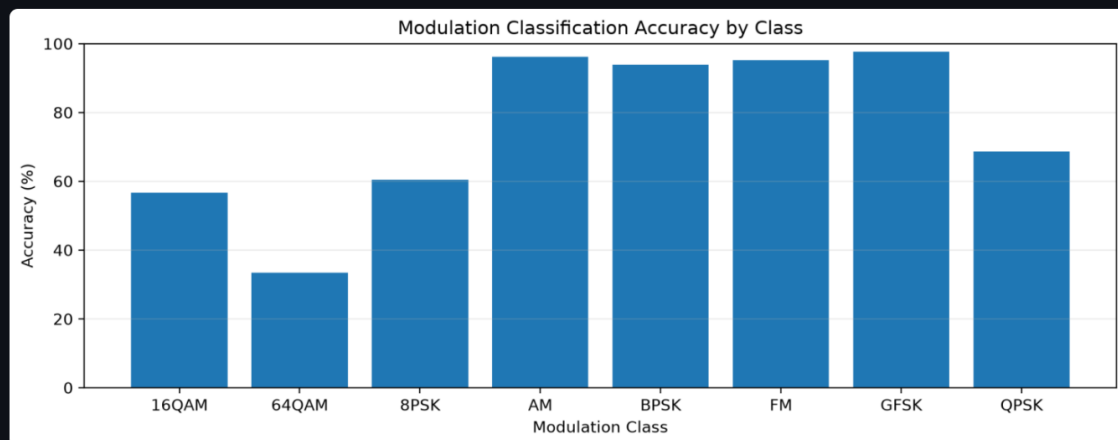
70.95%

Metrics calculated from the saved end-to-end evaluation dataset.

Modulation Accuracy vs SNR



Modulation Accuracy by Class



The dashboard includes a model-performance section based on the final end-to-end evaluation results.

The reported system performance is:

Metric	Result
Modulation Accuracy	75.24%
SNR MAE	0.47 dB
CFO MAE	689 Hz
Symbol-Rate Accuracy	70.95%

These values provide users with an overview of the measured performance of the complete system.

9.16 Modulation Accuracy versus SNR

The dashboard displays the relationship between SNR and modulation-classification performance.

The evaluated SNR levels are:

-10 dB
-5 dB
0 dB
5 dB
10 dB
15 dB
20 dB

The corresponding modulation accuracy values are:

SNR	Accuracy
-10 dB	56.61%
-5 dB	66.84%
0 dB	82.02%
5 dB	88.52%
10 dB	91.07%
15 dB	89.78%
20 dB	90.10%

This visualization demonstrates the relationship between signal quality and classification performance.

9.17 Modulation Accuracy by Class

The dashboard also provides class-wise modulation accuracy.

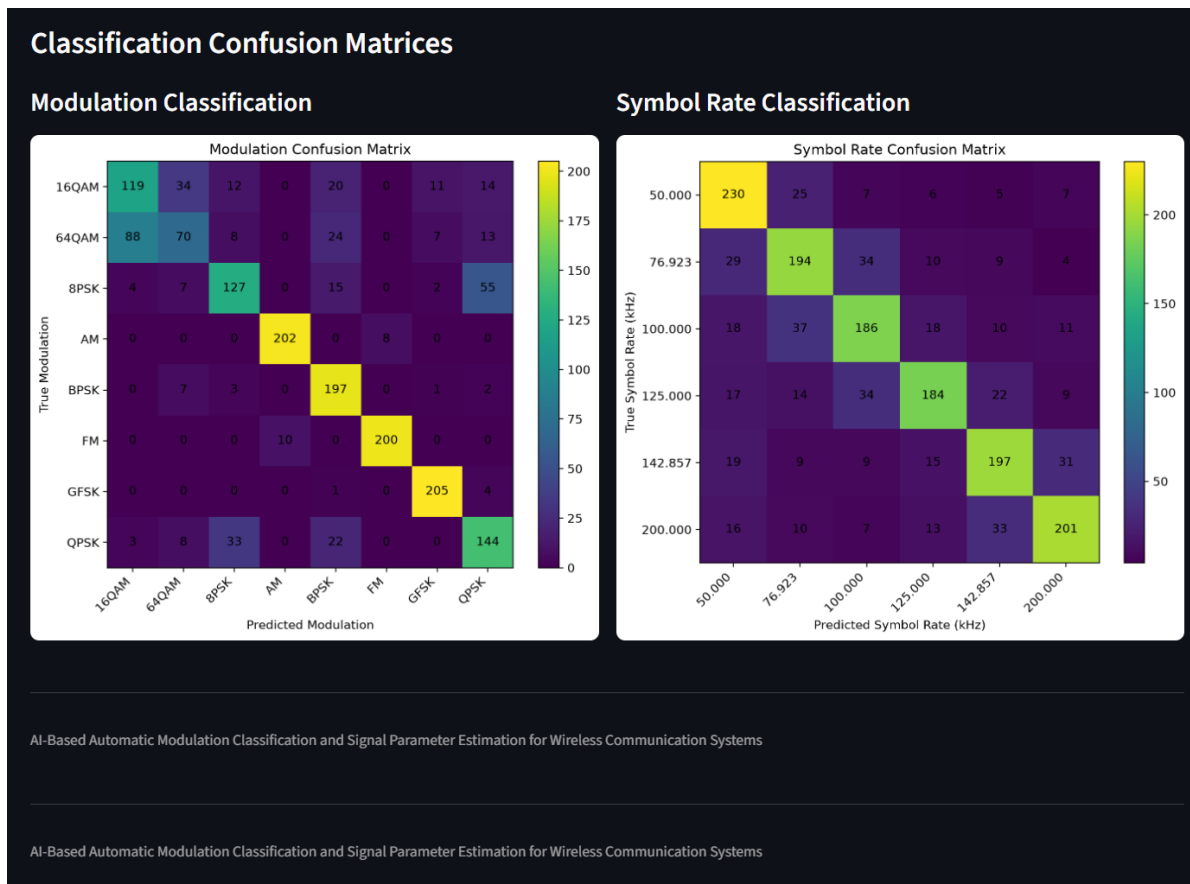
The evaluated values are:

Modulation	Accuracy
BPSK	93.81%
QPSK	68.57%
8PSK	60.48%

16QAM	56.67%
64QAM	33.33%
GFSK	97.62%
AM	96.19%
FM	95.24%

This view makes it possible to identify classes for which additional model improvement may be useful.

9.18 Confusion Matrices



The dashboard displays confusion matrices for:

1. Modulation classification
2. Symbol-rate classification

A confusion matrix represents the relationship between the true class and the predicted class.

For modulation classification, the matrix contains eight classes.

For symbol-rate classification, the matrix contains six supported rates.

These visualizations help identify systematic class confusion.

9.19 Analysis Report Export

The dashboard provides an **Export Analysis Report** feature.

The exported analysis can contain the principal information associated with the current signal analysis, including:

- Input information
- Predicted modulation
- Modulation confidence
- Estimated SNR
- Estimated CFO
- Estimated symbol rate
- Technical signal information

This allows the results of an individual signal analysis to be preserved for further use.

9.20 Dashboard Workflow Example

A typical analysis session follows these steps:

Step 1

Open the Streamlit dashboard.

Step 2

Select either:

Generate Synthetic Signal

or:

Upload I/Q Signal

Step 3

If generating a signal, select the desired signal parameters.

Step 4

If uploading a signal, select a supported `.npy`, `.npz`, or `.csv` file.

Step 5

Start the analysis.

Step 6

The system performs preprocessing and model inference.

Step 7

Review the AI predictions:

- Modulation
- Confidence
- SNR
- CFO
- Symbol rate

Step 8

Inspect the signal visualizations.

Step 9

Review the overall model-performance information.

Step 10

Export the analysis report if required.

9.21 Example Controlled Analysis

A controlled QPSK signal was used to verify the complete inference pipeline.

For one controlled test signal, the system produced:

Parameter	AI Output
Modulation	QPSK
Modulation Confidence	99.71%
SNR	10.00 dB
CFO	425.27 Hz
Symbol Rate	100.000 kHz
Samples	1024

This example demonstrates the operation of the complete inference pipeline for a generated signal.

The controlled spot-check is an example of system operation and should not be interpreted as the overall model accuracy. The 1,680-signal end-to-end evaluation remains the primary quantitative evaluation.

9.22 Dashboard Advantages

The developed dashboard provides several practical advantages:

Integrated Analysis

Multiple AI predictions and signal visualizations are available from one interface.

Interactive Input

Users can change signal parameters and immediately analyze the generated signal.

External Signal Support

Users can upload supported I/Q files for analysis.

Visual Interpretation

Waveforms, constellation diagrams, spectra, amplitude, and phase provide visual context for AI predictions.

Performance Transparency

The dashboard displays measured model-performance metrics rather than presenting predictions without evaluation information.

9.23 Dashboard Limitations

The current dashboard has several limitations.

First, the AI models were developed and evaluated primarily using controlled synthetic signals.

Second, the symbol-rate model supports only six predefined rates.

Third, the CNN uses a fixed primary analysis length of 1024 samples.

Finally, the accuracy of predictions on real-world over-the-air signals may differ from the controlled synthetic evaluation because practical wireless channels can introduce additional impairments.

9.24 Chapter Summary

The Streamlit dashboard provides the user-facing interface for the complete AI-based wireless signal analysis system.

It integrates:

- Signal generation
- I/Q file upload
- Input validation
- Modulation classification
- SNR estimation
- CFO estimation
- Symbol-rate classification
- Signal visualization
- Performance analysis
- Analysis-report export

The dashboard therefore converts the underlying machine-learning and signal-processing pipeline into an interactive application suitable for demonstration, experimentation, and educational use.

Chapter 10 — Limitations and Future Scope

10.1 Introduction

The developed system demonstrates the feasibility of combining artificial intelligence, machine learning, and digital signal processing for automatic modulation classification and wireless signal parameter estimation. However, the current implementation has several limitations.

Identifying these limitations is important because the reported performance was obtained under controlled experimental conditions. The limitations also provide directions for improving the system in future work.

10.2 Limitations

10.2.1 Synthetic Signal Dataset

The primary training and evaluation datasets were generated using the project's synthetic signal generator.

Although synthetic data provides controlled ground-truth values, real wireless signals may contain additional characteristics that are not completely represented in the current dataset.

Real-world signals may experience:

- Multipath propagation
- Fading
- Interference
- Timing errors
- Filtering effects
- Hardware imperfections
- Nonlinear distortion
- Frequency-selective channels

Therefore, performance on real over-the-air signals may differ from the controlled experimental results.

10.2.2 Modulation Classification Performance

The final modulation-classification accuracy was:

\[
75.24\%
\]

However, performance is not uniform across all eight modulation classes.

The class-wise results show lower performance for the higher-order QAM classes, particularly 64QAM.

This indicates that additional training data, improved signal modeling, and more advanced architectures could be investigated to improve discrimination between similar modulation schemes.

10.2.3 CFO Estimation

CFO estimation is currently the most challenging parameter-estimation component.

The final end-to-end CFO MAE was:

\[
688.56\ Hz
\]

The range-wise analysis showed larger errors near the boundaries of the approximately ± 3000 Hz operating range.

This indicates that the current feature-based regression approach does not maintain uniform estimation accuracy across the entire CFO range.

10.2.4 Limited Symbol-Rate Range

The symbol-rate model currently operates as a classification system.

It supports six predefined symbol-rate classes:

- 50 kHz
- 76.923 kHz
- 100 kHz
- 125 kHz
- 142.857 kHz

- 200 kHz

Consequently, the system cannot currently estimate an arbitrary continuous symbol rate outside the supported classes.

10.2.5 Fixed CNN Input Length

The primary CNN operates on 1024 samples.

Signals with substantially different observation lengths may therefore require segmentation, padding, or another preprocessing strategy before classification.

10.2.6 Computational Resources

The models were developed and trained in a CPU-based environment.

While this makes the project accessible without requiring a dedicated GPU, larger datasets and more complex deep-learning architectures could require substantially greater computational resources.

10.2.7 Controlled Channel Model

The current signal-generation process focuses on controlled signal conditions rather than a complete wireless propagation model.

More realistic channel conditions would provide a stronger test of system robustness.

10.3 Future Scope

Several improvements can be investigated in future versions of the project.

10.3.1 Real-World RF Dataset

The most important extension would be to incorporate real-world I/Q recordings obtained from software-defined radio hardware or publicly available RF datasets.

This would allow the models to be evaluated under practical signal conditions.

10.3.2 Advanced Channel Modeling

The synthetic generator can be extended to include:

- Rayleigh fading
- Rician fading
- Multipath propagation

- Doppler effects
- Timing offset
- Phase noise
- Frequency-selective fading
- Adjacent-channel interference

This would make the training and evaluation environment more representative of practical wireless systems.

10.3.3 Improved Deep-Learning Architectures

Future versions can investigate more advanced neural architectures, including:

- Deeper CNNs
- Residual networks
- CNN-LSTM architectures
- Transformer-based signal models
- Attention mechanisms

These architectures could potentially learn more complex relationships in I/Q sequences.

10.3.4 Improved CFO Estimation

The CFO estimator can be improved by combining machine learning with classical synchronization algorithms.

Possible approaches include:

- Higher-resolution spectral estimation
- Phase-difference analysis
- Higher-order transformations
- Coarse-to-fine CFO estimation
- Hybrid DSP and ML approaches

A hybrid estimator could potentially reduce the larger errors observed near the boundaries of the current CFO range.

10.3.5 Continuous Symbol-Rate Estimation

The current symbol-rate classifier can be extended into a regression-based estimator capable of predicting arbitrary symbol rates rather than selecting from six predefined classes.

This would increase the flexibility of the system when analyzing unknown communication signals.

10.3.6 Additional Modulation Schemes

Additional modulation formats can be incorporated, such as:

- 4FSK
- MSK
- GMSK
- OQPSK
- 32QAM
- 256QAM
- OFDM

Expanding the modulation library would increase the applicability of the AMC system.

10.3.7 Real-Time SDR Integration

The Streamlit application can be extended to receive live I/Q samples from software-defined radio hardware.

A future architecture could therefore be:

SDR Receiver → I/Q Stream → Preprocessing → AI Models → Real-Time Analysis Dashboard

This would transform the current offline analysis prototype into a real-time RF monitoring system.

10.3.8 Model Optimization

For deployment on resource-constrained systems, future work could investigate:

- Model quantization
- Pruning

- Knowledge distillation
- ONNX deployment
- Edge inference

These techniques could reduce memory and computation requirements.

10.3.9 Improved Uncertainty Estimation

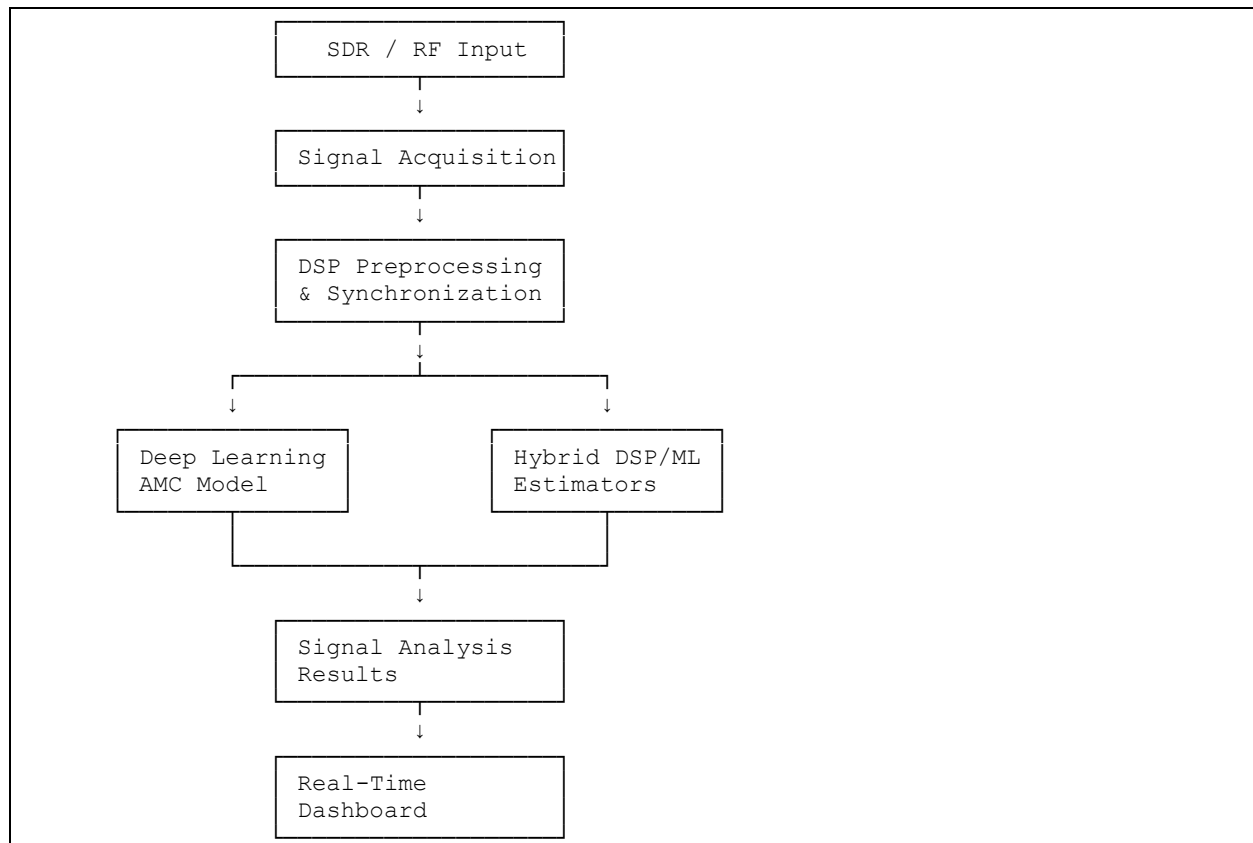
Future versions could provide more detailed uncertainty estimates rather than only classification confidence.

This could help distinguish between:

- High-confidence predictions
- Low-confidence predictions
- Out-of-distribution signals

10.4 Future System Architecture

A potential future version of the project can be extended as:



10.5 Chapter Summary

The developed system provides an integrated AI-based framework for modulation classification and signal parameter estimation. However, the current implementation is primarily evaluated using controlled synthetic signals and predefined parameter ranges.

Future development can focus on real-world RF data, realistic channel models, improved CFO estimation, continuous symbol-rate estimation, additional modulation classes, advanced neural architectures, and real-time SDR integration.

These improvements would extend the system from a controlled research prototype toward a more comprehensive wireless signal-analysis platform.

CHAPTER 11

CONCLUSION

11.1 Conclusion

This project presented the design and implementation of an **AI-Based Automatic Modulation Classification and Signal Parameter Estimation System for Wireless Communication Systems**. The primary objective was to develop an integrated software system capable of analyzing complex I/Q signals, identifying their modulation type, and estimating important signal parameters including Signal-to-Noise Ratio (SNR), Carrier Frequency Offset (CFO), and symbol rate.

The proposed system supports eight modulation classes: **BPSK, QPSK, 8PSK, 16QAM, 64QAM, GFSK, AM, and FM**. Synthetic complex I/Q signals were generated under controlled communication conditions with variations in modulation type, SNR, carrier frequency offset, and symbol rate. This provided a reproducible environment for training and evaluating the implemented machine-learning models.

For automatic modulation classification, a **Convolutional Neural Network (CNN)** was implemented to learn signal representations directly from the two-channel I/Q input. The final CNN achieved **81.10% test accuracy** on the common test dataset. In the controlled end-to-end evaluation involving 1,680 signals, the complete system achieved a modulation classification accuracy of **75.24%**.

The class-wise results demonstrated that classification performance varies considerably between modulation schemes. The system achieved high recognition rates for BPSK, GFSK, AM, and FM, while the more complex QAM and higher-order PSK classes presented greater classification difficulty. In particular, the end-to-end classification accuracies were 93.81% for BPSK, 68.57% for QPSK, 60.48% for 8PSK, 56.67% for 16QAM, 33.33% for 64QAM, 97.62% for GFSK, 96.19% for AM, and 95.24% for FM.

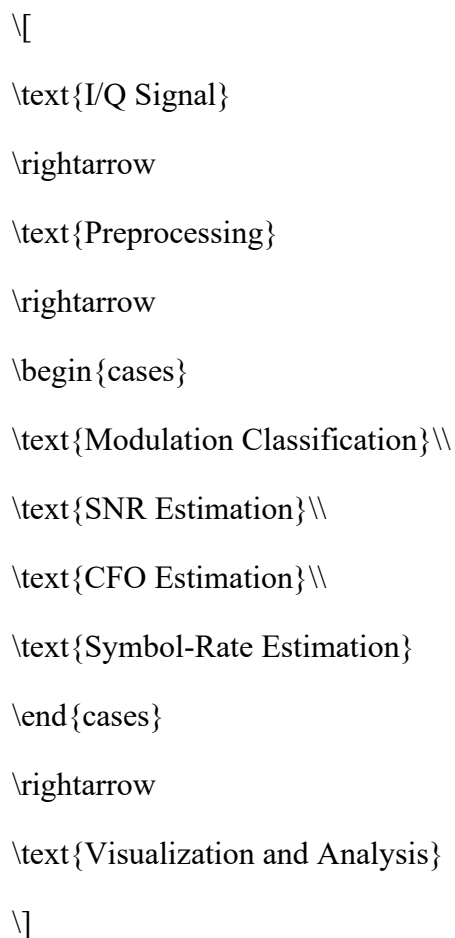
In addition to modulation classification, machine-learning-based parameter estimation models were developed. The final SNR estimation model achieved a **Mean Absolute Error (MAE) of 0.467 dB** and a **Root Mean Squared Error (RMSE) of 1.069 dB** during the end-to-end evaluation. These results demonstrate that the implemented approach can provide useful SNR estimates across the evaluated signal conditions.

For carrier frequency offset estimation, the final model achieved an end-to-end **MAE of approximately 688.56 Hz** and an **RMSE of approximately 1034.11 Hz**. The results also showed that estimation error varied with the magnitude of the frequency offset, with larger errors generally occurring toward the extreme ends of the evaluated CFO range.

Symbol-rate estimation was implemented as a classification problem over six supported symbol-rate categories. The complete system achieved approximately **70.95% symbol-rate classification accuracy**, with a mean absolute estimation error of approximately **15.29 kHz** in the end-to-end evaluation. The results indicate that symbol-rate estimation is feasible using the extracted signal characteristics, although additional improvements are required for more accurate continuous estimation.

A major component of the project was the development of an interactive **Streamlit-based dashboard**. The dashboard provides two signal-input modes: synthetic signal generation and external I/Q signal upload. It validates uploaded data and provides AI-based analysis along with signal visualizations including I/Q waveforms, constellation diagrams, frequency spectra, amplitude, and phase. The dashboard also displays model-performance information and confusion matrices, providing a convenient interface for examining the behavior of the complete system.

The project therefore demonstrates an integrated workflow:



The experimental results also identify several areas requiring further development. The current system relies primarily on synthetically generated signals and evaluates a fixed set of modulation

types and symbol-rate categories. Classification of higher-order modulation schemes, particularly 64QAM, remains challenging under the tested conditions. CFO estimation also shows substantially higher error than SNR estimation. Furthermore, the current symbol-rate estimator operates on predefined categories rather than providing unrestricted continuous symbol-rate estimation.

Despite these limitations, the project successfully demonstrates the integration of signal generation, digital signal processing, machine learning, deep learning, parameter estimation, automated inference, visualization, and software deployment into a single wireless-signal analysis platform.

11.2 Future Direction

The developed system provides a foundation for several future extensions. Real-world RF or software-defined-radio recordings can be incorporated to evaluate the system under practical propagation and hardware conditions. More realistic channel models can also be introduced, including multipath fading, Doppler effects, phase noise, timing offsets, and more complex interference conditions.

The modulation classifier can be further improved using larger datasets, advanced CNN architectures, residual networks, attention mechanisms, transformer-based models, or hybrid CNN-RNN architectures. Additional modulation schemes can also be incorporated to increase the coverage of the classifier.

Future work can also focus on improving CFO estimation through dedicated synchronization and frequency-estimation algorithms or hybrid signal-processing and machine-learning approaches. Symbol-rate estimation can be extended from discrete classification to continuous regression so that previously unseen symbol rates can also be estimated.

Finally, integration with a Software Defined Radio (SDR) platform would allow the system to process live RF signals and move the project from a controlled synthetic-data prototype toward a real-time wireless signal analysis system.

11.3 Final Statement

In conclusion, the developed system demonstrates that artificial intelligence and signal-processing techniques can be combined to perform automatic modulation classification and signal parameter estimation from complex I/Q signals. The implemented CNN and machine-learning models, together with the interactive dashboard, provide a complete experimental framework for wireless signal analysis.

The project establishes a functional foundation that can be extended with real-world RF data, improved channel models, more advanced learning architectures, additional modulation schemes, and real-time SDR processing in future work.

REFERENCES

- [1] T. J. O'Shea, J. Corgan, and T. C. Clancy, "Convolutional Radio Modulation Recognition Networks," in *Engineering Applications of Neural Networks (EANN)*, 2016, pp. 213–226. DOI: 10.1007/978-3-319-44188-7_16.
- [2] T. J. O'Shea and N. West, "Radio Machine Learning Dataset Generation with GNU Radio," *Proceedings of the GNU Radio Conference*, vol. 1, no. 1, 2016.
- [3] T. J. O'Shea, T. Roy, and T. C. Clancy, "Over-the-Air Deep Learning Based Radio Signal Classification," *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 168–179, 2018. DOI: 10.1109/JSTSP.2018.2797022.
- [4] T. Wang, G. Yang, P. Chen, Z. Xu, and Q. Ye, "A Survey of Applications of Deep Learning in Radio Signal Modulation Recognition," *Applied Sciences*, vol. 12, no. 23, 12052, 2022. DOI: 10.3390/app122312052.
- [5] S. Haykin, *Communication Systems*, 4th ed. New York, NY, USA: Wiley, 2001.
- [6] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York, NY, USA: McGraw-Hill, 2008.
- [7] S. O. Rice, "Mathematical Analysis of Random Noise," *Bell System Technical Journal*, vol. 23, no. 3, pp. 282–332, 1944.
- [8] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ, USA: Wiley-Interscience, 2006.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [12] W. McKinney, *Python for Data Analysis*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [13] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," 2016. arXiv:1603.04467.
- [14] Streamlit, "Streamlit Documentation," Streamlit Inc., documentation for the framework used to develop the interactive signal-analysis dashboard.

APPENDIX

APPENDIX A — PROJECT DIRECTORY STRUCTURE

The final project is organized into modules for signal generation, preprocessing, model training, inference, testing, visualization, and dashboard deployment.

```
amc_project/
├── src/
│   └── amc/
│       ├── signal_generator.py
│       ├── preprocessing.py
│       ├── estimators.py
│       ├── feature_extractors.py
│       ├── train.py
│       ├── inference.py
│       ├── cnn.py
│       └── cnn_dataset.py
├── scripts/
│   ├── evaluate_end_to_end.py
│   ├── plot_results.py
│   ├── validate_all_modulations.py
│   ├── validate_snr.py
│   ├── validate_cfo.py
│   └── validate_symbol_rate.py
├── dashboard/
│   └── app.py
├── models/
│   ├── cnn_common.pth
│   ├── snr_parameter_rf.joblib
│   ├── cfo_parameter_rf.joblib
│   ├── symbol_rate_classifier.joblib
│   └── end_to_end_results.csv
├── data/
│   └── datasets
├── tests/
│   └── test_amc.py
├── results/
│   └── evaluation_plots
```

APPENDIX B — SUPPORTED MODULATION CLASSES

The system supports the following eight modulation classes:

No.	Modulation
1	BPSK
2	QPSK
3	8PSK
4	16QAM

5	64QAM
6	GFSK
7	AM
8	FM

APPENDIX C — EVALUATED SIGNAL PARAMETERS

Parameter	Configuration
Number of modulation classes	8
Signal length	1024 samples
Sample rate	1 MHz
SNR range	−10 to 20 dB
Number of SNR levels	7
CFO range	Approximately −3000 to +3000 Hz
Symbol-rate categories	6
Signal representation	Complex I/Q
Main classifier	CNN
Parameter models	Random Forest / classifier

APPENDIX D — FINAL MODEL PERFORMANCE

Task	Final Result
CNN test modulation accuracy	81.10%
End-to-end modulation accuracy	75.24%
SNR MAE	0.467 dB
SNR RMSE	1.069 dB
CFO MAE	688.56 Hz
CFO RMSE	1034.11 Hz
Symbol-rate accuracy	70.95%
Symbol-rate MAE	15.29 kHz

APPENDIX E — CLASS-WISE MODULATION RESULTS

Modulation	End-to-End Accuracy
BPSK	93.81%
QPSK	68.57%
8PSK	60.48%
16QAM	56.67%
64QAM	33.33%
GFSK	97.62%
AM	96.19%
FM	95.24%

APPENDIX F — EXAMPLE SYSTEM OUTPUT

For a controlled QPSK test signal, the system produced an example output containing:

Modulation	: QPSK
Confidence	: 99.71%
Estimated SNR	: 10.00 dB
Estimated CFO	: 425.27 Hz
Symbol Rate	: 100.000 kHz
Sample Rate	: 1 MHz
Number of Samples	: 1024

This example demonstrates the complete inference output generated by the system.

APPENDIX G — TESTING

The project includes automated tests for core signal-processing functionality.

The implemented test suite was executed using `pytest`, with the project test suite producing:

3 passed

Additional controlled validation scripts were used to test:

- All eight modulation classes
- SNR estimation
- CFO estimation
- Symbol-rate estimation
- End-to-end system performance

APPENDIX H — GENERATED RESULTS AND VISUALIZATIONS

The project generates the following major visualization outputs:

1. CNN Accuracy vs SNR
2. CNN Confusion Matrix
3. End-to-End Modulation Confusion Matrix
4. Modulation Accuracy by Class
5. SNR MAE vs SNR
6. CFO MAE by CFO Range
7. Symbol-Rate Confusion Matrix
8. Final System Performance

These figures are included in the Results and Discussion chapter and may also be provided as supplementary material.

APPENDIX I — SOFTWARE TOOLS

The major software tools and libraries used in the project include:

- Python
- NumPy
- SciPy
- Pandas
- Scikit-learn
- PyTorch
- Matplotlib
- Joblib
- Streamlit
- Pytest
- Git
- Git LFS